
MEMORIAS DEL CURSO DE ANÁLISIS Y DISEÑO ORIENTADOS POR OBJETOS (ADOO)

Gabriel Mañana Guichón

Departamento de Sistemas
Facultad de Ingeniería
Universidad Nacional de Colombia

"... the key to program quality is simplicity: simple models, simple designs, simple code.
If a problem is really understood, a simple solution can be found."

Senter for Industriforskning (?)

Introducción

El presente documento está basado en el artículo "Síntesis de las diferentes metodologías utilizadas en Análisis y Diseño Orientados por Objetos (ADOO)" del mismo autor. En este artículo (que se anexa al presente trabajo), se hace una breve revisión de la crisis actual del desarrollo de software, se hace una síntesis de las metodologías planteadas por los autores más relevantes en el área y se estudia en particular la metodología propuesta por el grupo de investigadores compuesto por Rebecca Wirfs-Brock, Brian Wilkerson y Lauren Wiener. Si bien en el artículo referido se lleva a cabo un análisis teórico, donde se pretende identificar los componentes elementales de una metodología OO que permitan realizar un diseño consistente y completo, en este trabajo se pretende presentar una versión combinada y 'atterrizada' de las metodologías existentes, fruto de la experiencia del autor en el desarrollo de aplicaciones gráficas interactivas. La metodología presentada está basada mayormente en la propuesta de Wirfs-Brock et al. en [Wirfs-Brock, 90] y en diferentes ideas planteadas por B. Meyer en [Meyer, 88] y por P. Coad y E. Yourdon en [Coad, 92]. Después de revisar las metodologías tradicionales y de presentar los conceptos básicos involucrados en este nuevo paradigma orientado por objetos, se comienza con el diseño de una aplicación de ejemplo, diseño que se irá desarrollando a través del curso a medida que se van presentando las actividades a realizar.

La elección de una aplicación gráfica interactiva como ejemplo de aplicación responde a varias consideraciones. En primer lugar, y lógicamente, porque cae en el área de investigación y trabajo (experiencia) del conferencista. En segundo lugar porque las exigencias de software actuales (partiendo de los mismos sistemas operativos) hacen que casi todo programa sea interactivo y provea una interfaz gráfica. En tercer lugar, y quizás la razón más importante, porque un editor gráfico orientado al manejo de objetos (no bitmaps) por elemental que sea, plantea los desafíos típicos de toda aplicación interactiva: una interfaz con el usuario compleja que implica interacción con varias ventanas (ventana de edición, ventana de herramientas, ventana de colores, etc.), procesamiento de eventos de mouse, teclado y del propio sistema operativo, persistencia del modelo, undo/redo de múltiples niveles, soporte de clipboard, impresión, etc. Es un buen ejemplo para probar la metodología planteada, pero sobre todo es un buen ejemplo para evaluar el impacto causado por cambios en las especificaciones (extensión en este caso), área donde es particularmente fuerte el paradigma orientado por objetos.

El documento está organizado en las siguientes secciones:

1. En la primera sección se hace una revisión del ciclo de vida tradicional de los sistemas de software. Se revisa el enfoque planteado en la Descomposición Funcional y en el Desarrollo Estructurado de Jackson, y se presenta de forma introductoria el enfoque orientado por objetos.
2. En esta sección se presentan los conceptos básicos involucrados en este nuevo paradigma: abstracción, objetos, ocultamiento de información, relación cliente-servidor, clases de objetos, polimorfismo y herencia. Se presenta también una primera visión de la metodología propuesta, que consta de dos etapas: fase exploratoria y fase analítica.
3. Aquí se presentan los pasos a seguir para encontrar las clases de objetos que componen una aplicación. Se inicia la identificación de las clases de la aplicación de ejemplo a partir de la especificación de requerimientos.
4. En esta sección se define el propósito de las clases identificadas, asignándoles conocimiento y comportamiento, esto es, determinando las responsabilidades de cada una. La metodología propuesta en este curso está basada en identificar las responsabilidades de cada una de las clases que conforman la aplicación.
5. Las clases cumplen con sus responsabilidades de dos maneras: llevando a cabo las tareas necesarias por si mismas o con la colaboración de otras clases. En las secciones anteriores se presentaron los lineamientos para identificar las clases que componen una aplicación y algunas guías para la asignación de responsabilidades. Ahora se determina la forma en que las clases interactúan para cumplir con sus responsabilidades.
6. A partir de la información obtenida en la primera fase se propone comenzar la fase analítica del diseño. El fin de este análisis está en obtener un nivel de comprensión más global del sistema que el obtenido en la fase exploratoria. Para esto se cuenta con las siguientes herramientas conceptuales y gráficas: grafos de jeraquías, diagramas de Venn, y contratos entre las clases.
7. Los pasos anteriores están encaminados a producir un diseño inicial. Se identifican las clases del sistema, sus responsabilidades y las colaboraciones que garantizan los servicios requeridos. Se analizan las relaciones de herencia entre las clases y se identifican los contratos soportados. En esta última etapa se refina el diseño obtenido simplificando los patrones de comunicación entre las clases. Se dividen las responsabilidades en grupos de clases llamados *subsistemas* y se mejora la forma en que estos encapsulan la funcionalidad global.
8. En esta sección se muestra como utilizar la información generada por los procesos anteriores para producir la especificación final de diseño.

9. Por último se presentan algunas consideraciones de implantación, tales como la elección de un lenguaje de programación en particular, evaluación e implementación del diseño realizado, y verificación del programa construido.

1. Ciclo de vida tradicional de los sistemas de software

En ingeniería de software, la descripción tradicional del ciclo de vida del software está basada en un modelo conocido como el modelo de cascada (*waterfall model*). Inicialmente el modelo planteaba la discretización de las actividades involucradas en el desarrollo de software y las presentaba como una serie lineal de tareas, donde cada tarea debía ser completada antes de comenzar la siguiente (figura 1).

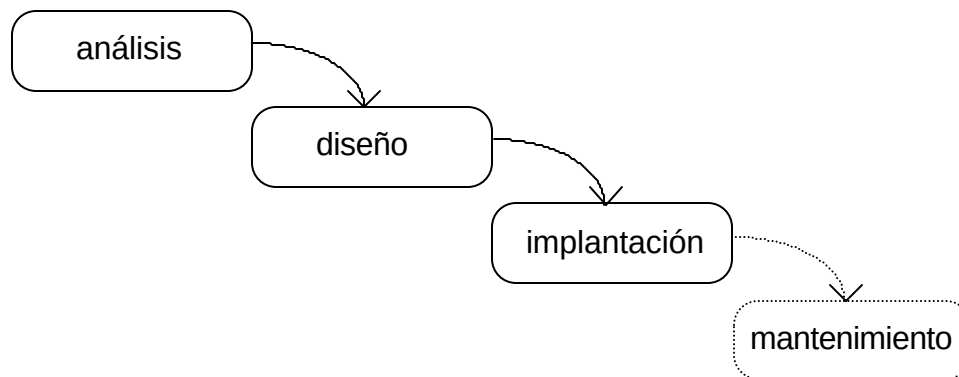


Fig. 1. Ciclo de vida tradicional

Frente al hecho de que en la realidad no es posible lograr este grado de completitud, posteriores variaciones del modelo reconocen la necesidad de iterar a través de las diferentes etapas (figura 2).

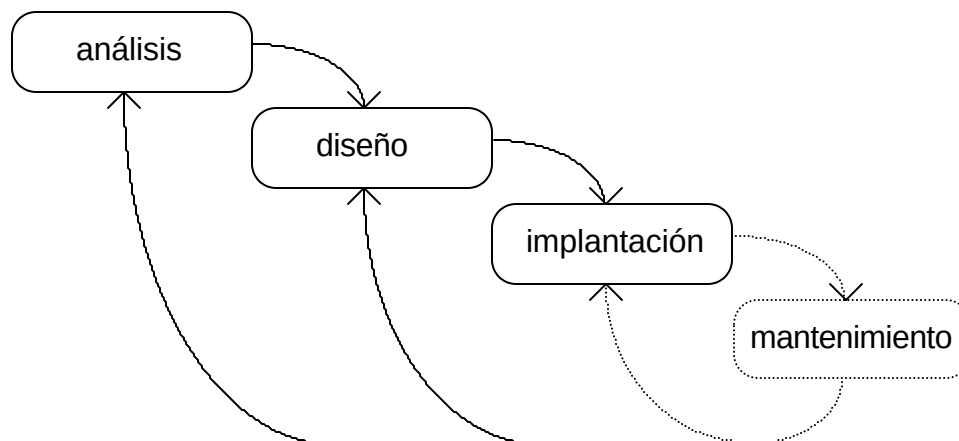


Fig. 2. Modelo de cascada revisado

Todos los autores en el área coinciden en cuatro etapas: análisis, diseño y construcción. La fase de análisis cubre básicamente el análisis de los requerimientos del usuario (dominio del problema) y un estudio de factibilidad/viabilidad. La fase de diseño cubre los diferentes conceptos del diseño

de sistemas (dominio de la solución): diseño global o diseño lógico y diseño del programa, conocido como diseño detallado o diseño físico. Una vez diseñada la aplicación se escribe el código correspondiente y se pone a prueba el programa. Aquí comienza la fase de mantenimiento donde se corrigen fallas y se adecúa el programa a nuevos requerimientos. A un nivel más detallado se identifican nuevas actividades tal como se ve en la siguiente figura.

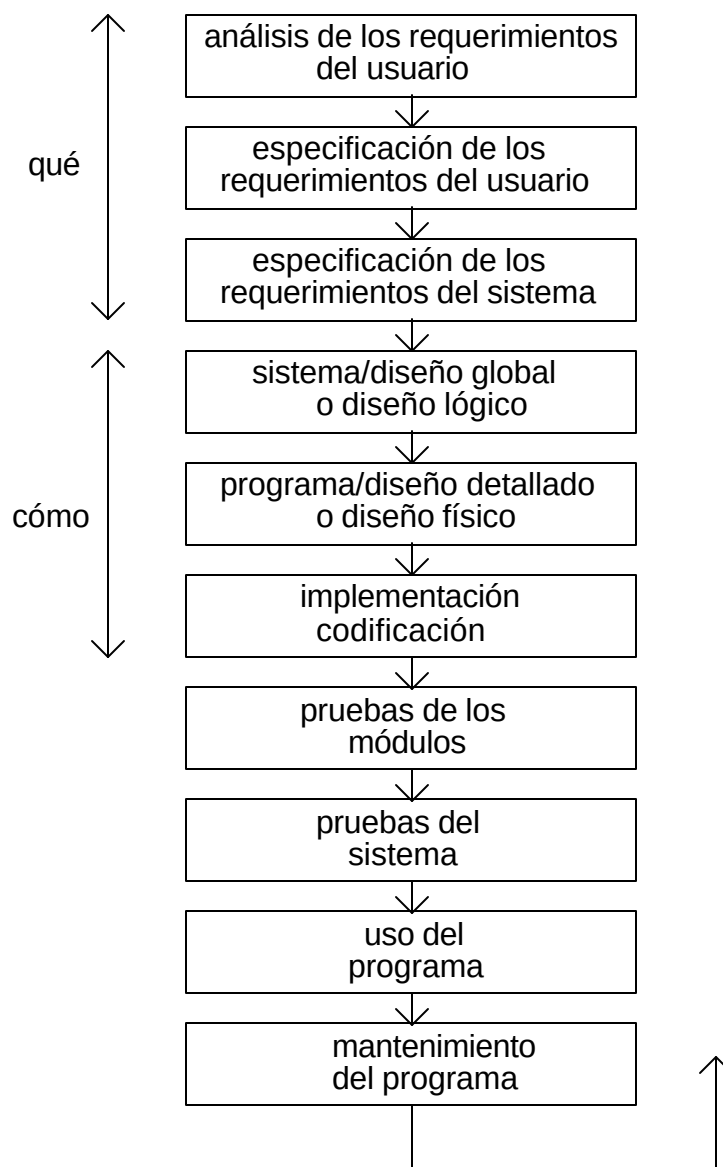


Fig. 4. Modelo de cascada a nivel detallado

Se comienza con la definición del problema y se identifican los requerimientos de los usuarios actuales y futuros, usualmente por medio de preguntas, discusión con expertos en el área, etc.. Dependiendo de la importancia y volumen del sistema esta fase puede incluir un estudio de

factibilidad/viabilidad. A partir de esto se escriben sendos documentos con las especificaciones de requerimientos del usuario y con las especificaciones de requerimientos de la aplicación. La especificación de requerimientos del usuario se escribe en el lenguaje de los usuarios, lo que permite establecer un acuerdo sobre esta especificación entre los usuarios del sistema y los ingenieros del sistema. La especificación de requerimientos de la aplicación se escribe en el lenguaje de los programadores y establece en forma precisa los detalles del sistema. Estas actividades comprenden el análisis del espacio del problema (qué). A medida que avanzamos en el diagrama pasamos del análisis al diseño del sistema, esto es, del espacio del problema al espacio de la solución.

La etapa de diseño es la que está definida en forma más imprecisa, quizás porque es un proceso creativo, no mecánico, donde se hace una descomposición cada más detallada y progresiva del sistema. El diseño de sistemas es generalmente referido como "diseño global" o "diseño lógico" y el diseño del programa como "diseño detallado" o "diseño físico". En el ciclo de vida tradicional estas dos actividades de diseño pueden llegar a traslaparse y ser realizadas en forma iterativa. En el ciclo de vida orientado por objetos la diferenciación desaparece casi totalmente.

El ciclo de vida del software, tal como se ha descrito en los párrafos anteriores, es frecuentemente implementado basándose en una visión del mundo interpretada en términos de **descomposición funcional**, donde la pregunta fundamental a responder mediante las etapas de análisis y diseño es **qué** hace el sistema, esto es, cuál es su función. De esta manera, el diseño funcional (y las técnicas de descomposición funcional utilizadas para llevarlo a cabo) está basado en una interpretación funcional del espacio del problema y su traducción al espacio de la solución como un grupo de procedimientos interrelacionados. El sistema final es visto entonces como un conjunto de funciones que operan sobre los datos, aparentemente secundarios.

La descomposición funcional es también una metodología de análisis y diseño arriba-abajo (*top-down*) y es, en consecuencia, demasiado restrictiva para dar soporte al diseño de las aplicaciones contemporáneas. Meyer [Meyer] resume las desventajas de la metodología de diseño arriba-abajo de la siguiente forma:

1. La metodología arriba-abajo no tiene en cuenta el carácter evolutivo de los sistemas (i.e. cambio en las especificaciones, extensión, ...).
2. En la metodología arriba-abajo el sistema está caracterizado por una sola función, un concepto cuestionable.
3. En la metodología arriba-abajo el punto de vista es básicamente funcional y por lo tanto las estructuras de datos que el sistema manipula son dejadas en un plano secundario.
4. La metodología arriba-abajo no promueve la reutilización. (!)

Hemos hecho una revisión del enfoque tradicional del ciclo de vida del software, una metodología básicamente funcional. Ahora haremos una breve descripción de las tres opciones principales para el análisis y diseño de sistemas.

Descomposición Funcional

Según esta metodología, el diseñador debe centrar su atención en los procedimientos y algoritmos necesarios para que el sistema lleve a cabo su tarea. Inicialmente, el sistema es visto en términos de lo *que* debe hacer y luego en la etapa de diseño se determina *como* lo va a hacer. Las herramientas de diseño que dan soporte a esta metodología están basadas en el *flujo de datos* de la aplicación, e incluyen los Diagramas de Flujo de Datos (DFDs), los Diccionarios de Datos (DDs) y los Diagramas de Estructuras.

Existen varios lenguajes de programación que dan soporte a la descomposición funcional: Fortran, Cobol, Pascal, C, etc.. La tendencia en estos lenguajes de colocar los datos compartidos en áreas globales (e.g. *COMMON* en Fortran, *extern* en C) produce los llamados 'efectos colaterales' (*stack of dominoes*), tan conocidos por aquellos que trabajan en el desarrollo y mantenimiento de programas, donde los cambios hechos en una parte del sistema es fuente de problemas en otras áreas aparentemente disociadas. Por otra parte, un sistema diseñado con una única función en mente, difícilmente podrá asimilar los cambios evolutivos causados por modificaciones o extensiones.

Desarrollo Estructurado de Jackson

La Programación Estructurada de Jackson (Jackson Structured Programming) y el Desarrollo Estructurado de Jackson (JSD, Jackson Structured Development) [Jackson] son técnicas que modelan el mundo real con un enfoque parcialmente orientado por objetos, donde se utilizan métodos basados en las estructuras de datos. Sin embargo, son esencialmente metodologías de descomposición funcional [Sommerville, p.185] en donde las estructuras de datos son utilizadas como soporte a esta descomposición. A diferencia de la descomposición funcional en donde las estructuras de datos están fundamentalmente determinadas por la estructura funcional, se puede decir que JSD es una metodología parcialmente orientada por objetos o de centro-afuera (*middle-out*), que hace menos énfasis en la parte funcional y comienza el análisis del sistema como un proceso de modelamiento.

Desarrollo Orientado por Objetos

El paradigma Orientado por Objetos (OO) utiliza los mismos componentes de los sistemas de software: datos y procedimientos. Sin embargo, desenfatiza los procedimientos y hace énfasis en el encapsulamiento de datos y procedimientos en entidades con una interfaz claramente definida. En la descomposición de sistemas basada en un enfoque OO, el sistema es visto como una colección de entidades (objetos) que encapsulan cierto conocimiento (datos) y cierto comportamiento (procedimientos), y que interactúan para realizar la tarea o tareas del caso. Las

etapas de análisis y diseño se llevan a cabo con base en estos objetos y en los servicios que éstos prestan, utilizando un modelo cliente-servidor donde los objetos interactúan por medio de mensajes. El uso de este modelo cliente-servidor hace que el sistema sea descrito como 'orientado a responsabilidades' (*responsibility-driven*). El diseño detallado, incluyendo la implementación de procedimientos y definición de estructuras de datos, se difiere hasta una etapa más avanzada del proceso de desarrollo y es privada a cada clase de objetos, adhiriendo estrictamente al concepto de ocultamiento de información. De esta forma los procedimientos y estructuras de datos no quedan 'congelados' desde las primeras etapas del diseño, cuando todavía no se tiene un conocimiento profundo del sistema. Un sistema representado en función de objetos es por lo tanto más flexible y a la vez menos sensible a los cambios estructurales. Es importante que las estructuras de datos no sean especificadas muy temprano en el proceso de diseño. En consecuencia el desarrollo orientado por objetos se centra en la abstracción de datos en vez de fijar una estructura determinada en la especificación de los objetos que componen el sistema.

En contraste con el enfoque de descomposición funcional arriba-abajo, el enfoque OO si bien tiene atributos arriba-abajo está caracterizado por una metodología abajo-arriba, donde el fin principal está en generar clases de objetos que puedan ser incluidas en bibliotecas de uso general. Un enfoque que involucra una técnica de análisis arriba-abajo y una técnica de diseño abajo-arriba es probable que dé como resultado sistemas más robustos. En esta metodología la línea divisoria entre las diferentes etapas del ciclo de vida del software (A/D/I) desaparecen, tornándose un proceso continuo e iterativo donde analistas, diseñadores y programadores trabajan con entidades comunes: objetos.

Por último, una de las características que diferencian radicalmente la metodología OO de la metodología tradicional basada en la descomposición funcional está en los objetivos de cada una. Mientras que en el enfoque tradicional el fin es construir un sistema que de solución a un problema determinado (y este sistema se construye desde cero), en el enfoque OO el fin principal está en generar clases de objetos, que en conjunto den la solución requerida, pero que además puedan ser incluidas en bibliotecas para su futura **reutilización**. La reutilización de componentes probados y optimizados es lo que marca la verdadera diferencia entre las metodologías y se traduce en un aumento considerable de la productividad.

2. Conceptos básicos

Hoy en día las aplicaciones son demasiado voluminosas y complejas para ser manejadas por una sola persona. Las aplicaciones de software son complejas porque modelan la complejidad del mundo real. La cuestión importante para un diseñador es ¿Cómo manejar tal complejidad?

Abstracción. Proceso mediante el cual se ignoran los aspectos de un asunto que no son relevantes para el propósito en cuestión, con el fin de concentrar la atención en los aspectos más importantes. [Oxford, 1986], tomado de [Coad].

La abstracción como un proceso mental natural

La gente comprende el mundo a través de la construcción de modelos mentales de las partes del mundo de interés. El modelo mental de alguna cosa es una vista simplificada de como esta funciona, con el fin de interactuar con ella. En esencia, el proceso de construcción de modelos mentales es el mismo que el proceso de diseño de software, con la diferencia de que el modelo producido por el diseño de software debe ser manipulado por un computador. La abstracción es esencial en el funcionamiento de la mente y es la herramienta más poderosa con que contamos para el manejo de la complejidad.

La abstracción en el desarrollo de software

La abstracción es la clave en el diseño de buen software. Los objetivos de las aplicaciones de hoy en día son mucho más ambiciosos de lo que solían ser, debido mayormente a que ahora somos capaces de hacer mucho más de lo que solíamos. Parece un juego de palabras, pero no lo es. Y ahora somos capaces de hacer mucho más que antes porque hemos construido las abstracciones necesarias.

En los primeros días de la computación, los programadores enviaban instrucciones binarias al computador abriendo o cerrando interruptores en su panel frontal. Los nemónicos (instrucciones) del lenguaje ensamblador fueron abstracciones creadas para liberar a los programadores de recordar las secuencias de bits que componían cada instrucción. El siguiente paso en el uso de la abstracción aparece con la posibilidad de agrupar secuencias de instrucciones elementales en instrucciones definibles por el programador llamadas macro-instrucciones. Más adelante, los lenguajes de alto nivel le permiten al programador tomar distancia de la arquitectura específica de un determinado procesador o equipo. Esta abstracción hizo posible la generación de programas de propósito general y, hasta cierto punto, sin tener en cuenta el equipo destino. De la misma forma en que se puede evocar un conjunto de instrucciones de procesador por medio de un nombre, macro o instrucción de alto nivel, estas instrucciones de alto nivel pueden ser agrupadas en procedimientos que pueden ser evocados en una sola instrucción. La programación estructurada promueve entonces el uso de abstracciones de control, ciclos o sentencias if-then-else, permitiéndole a los programadores cambiar la secuencia de ejecución.

Recientemente, los tipos abstractos de datos (ADTs) le han permitido a los programadores escribir código sin preocuparse por la forma específica en que los datos son representados. El programador trabaja ahora a un nivel de abstracción donde especifica lo que se puede hacer con los datos. Los detalles de representación interna están ocultos y son considerados detalles de bajo nivel que no conciernen a los diseñadores del sistema. Por ejemplo, una *pila* puede ser definida en forma abstracta como una colección de elementos con un comportamiento LIFO (Last In-First Out). A partir de esta definición se pueden especificar las operaciones que se pueden realizar sobre una *pila* sin tener que especificar si los elementos son almacenados físicamente en un arreglo, una lista encadenada u otra estructura.

La metodología orientada por objetos descompone los sistemas en objetos. Estos son los componentes básicos del diseño y su utilización provee nuevos mecanismos de abstracción.

Objetos

El enfoque orientado por objetos aborda el manejo de la complejidad de todo sistema en el mundo real, haciendo una abstracción de su conocimiento y encapsulándolo en objetos. Acostumbramos dividir la información en datos y funciones. Este enfoque, llamado procedimental o descomposición funcional, aborda primero las tareas a llevar cabo y las va descomponiendo en sub-tareas hasta llegar al nivel de instrucciones directamente codificables en el lenguaje de programación. El enfoque orientado por objetos descompone el sistema en un conjunto de entidades que saben jugar su papel dentro del sistema. Cada entidad es responsable de una parte del conocimiento total del sistema: debe ser capaz de 'recordar' ciertos datos y debe ser capaz de 'realizar' ciertas operaciones. Esta práctica de agrupar datos y procedimientos (operaciones sobre los datos) en una entidad es conocida como *encapsulamiento*. El encapsulamiento ayuda en el manejo de la complejidad inherente de todo sistema particionando y distribuyendo esta complejidad en entidades independientes: objetos.

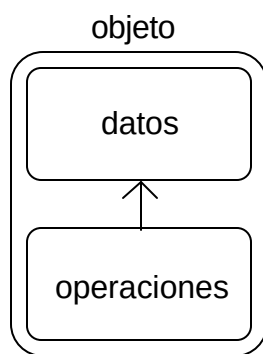


Fig. 5. Encapsulamiento.

Por medio del encapsulamiento se puede entonces atacar cada objeto por separado, analizar, diseñar, implementar y probar/depurar cada uno como una unidad independiente. Este es un factor importante cuando se trata de grupos de trabajo donde cada grupo desarrolla un módulo (o sub-sistema) del sistema. Como veremos más adelante, lo anterior tiene implicaciones importantísimas en la etapa de mantenimiento.

Ocultamiento de información

Ahora bien, un conjunto de objetos totalmente aislados unos de otros sería inútil en la construcción de un sistema. De aquí que cada objeto tenga una '*interfaz pública*' por medio de la cual se comunica con los otros objetos de la aplicación y una '*representación privada*' del conocimiento que encapsula. Estas 'facetas' del objeto están claramente diferenciadas y el principio aplicado se conoce como '*ocultamiento de información*'.

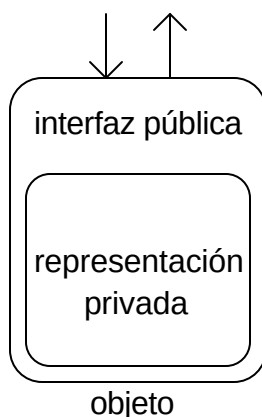


Fig. 6. Ocultamiento de información.

El principio de ocultamiento de información diferencia la habilidad de realizar alguna acción de los pasos específicos que se deben seguir para realizarla. Los objetos de la aplicación saben que operaciones pueden requerir a los otros objetos sin preocuparse de como se llevan a cabo. De esta manera el diseñador puede trabajar a un nivel de abstracción tal que le permita diferir los detalles de implementación y concentrarse en la arquitectura del sistema. El encapsulamiento y el ocultamiento de información permiten cambiar la representación interna de un objeto (por optimización, extensión, etc.) sin que aparezcan efectos secundarios en el resto de la aplicación¹. Los objetos son el medio de poner en práctica estos dos principios en desarrollo de programas.

Mensajes

Los objetos son accedidos a través de su interfaz pública por medio de *mensajes*. Un objeto accesa otro objeto de la aplicación enviándole un mensaje donde se le pide que lleve a cabo una

¹ Siempre que se mantenga invariable su interfaz pública.

operación o que provea cierta información. Un mensaje consiste de un nombre y eventualmente un conjunto de argumentos (al igual que una clásica llamada a función).

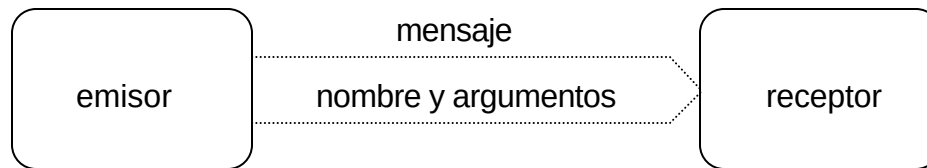


Fig. 7. Mensajes.

Cuando un objeto recibe un mensaje realiza la operación solicitada por medio de la ejecución de un *método* cuyo nombre coincide con el nombre del mensaje. El conjunto de mensajes al cual responde un objeto es conocido como el *comportamiento* del objeto y los datos que encapsula como sus *atributos*. Ajustando el esquema de objeto introducido en la figura 5 a esta terminología tenemos:

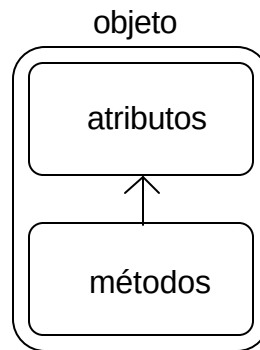


Fig. 8. Composición de un objeto.

Clases

Los objetos que comparten un mismo conjunto de atributos y un mismo comportamiento pertenecen a una misma *clase*. De esta forma, una clase es la especificación genérica de un conjunto arbitrario de objetos similares. Los objetos creados a partir de una clase en particular son llamados *instancias* de esa clase. El concepto de clase puede ser asimilado como la evolución del concepto de tipo dentro de un contexto clásico, así como los objetos pueden asimilados como una evolución de las variables.

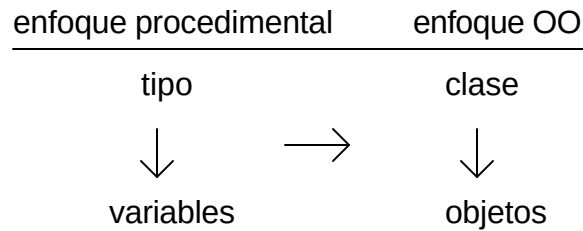


Fig. 9. Evolución del concepto de tipo.

Polimorfismo

El polimorfismo es otro mecanismo de abstracción y se basa en la posibilidad de que dos o más clases de objetos respondan a un mismo mensaje de maneras diferentes. Esto significa que el objeto que en tiempo de ejecución envía un mensaje no necesita saber exactamente a que objeto le está enviando el mensaje, sino que le es suficiente saber que existen varias clases de objetos capaces de responder a ese mensaje. El polimorfismo nos permite reconocer y explotar la similitud existente entre diferentes clases de objetos.

Herencia

Otro mecanismo de abstracción soportado por los lenguajes de programación es la *herencia*. La herencia es el mecanismo que permite a una clase definir su conocimiento (atributos y comportamiento) como extensión del conocimiento de otra clase o clases, esto es, 'heredando' su conocimiento. Nuevas clases pueden ser definidas a partir de otras existentes con solo especificar las diferencias, sin tener que construirlas de cero. Este mecanismo de especialización es la herramienta más poderosa que provee la metodología OO ya que promueve la reutilización y esta se traduce en un aumento considerable en la productividad.

La herencia permite también la clasificación taxonómica de las clases en jerarquías, un mecanismo que siempre ha ayudado en el estudio de la naturaleza. En esta clasificación la clase que hereda es llamada *subclase* y la clase heredada *superclase*. Las superclases son también llamadas clases ancestras y las subclases clases descendientes.

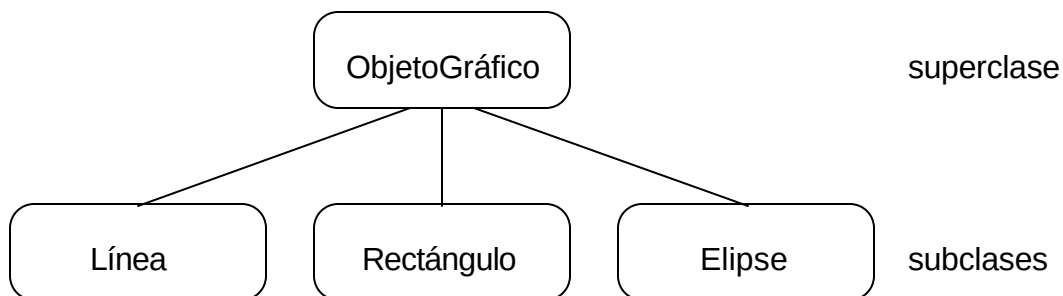


Fig. 10. Jerarquía de herencia.

Los lenguajes de programación OO dan soporte al mecanismo de herencia en dos modalidades: herencia simple y herencia múltiple. En la herencia simple una superclase puede tener varias subclases pero cada subclase solo tiene una superclase. En la herencia múltiple una subclase pueden heredar de más una superclase.

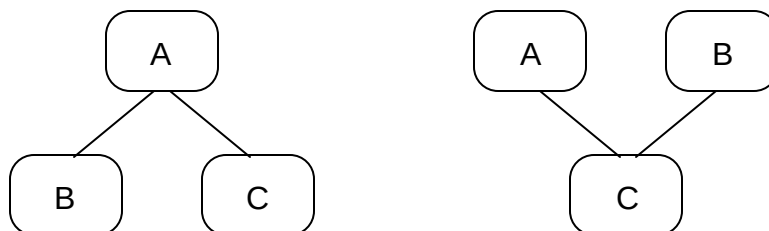


Fig. 11. Herencia simple y herencia múltiple.

Clases abstractas

Algunas clases no son generadas para crear objetos a partir de ellas, sino con el fin de agrupar atributos y comportamiento común a un conjunto de subclases. Estas clases especifican su comportamiento pero no lo implementan completamente y por esta razón son llamadas *clases abstractas*. Los métodos especificados en una clase abstracta deben ser redefinidos en las clases descendientes (*clases concretas*) a partir de las cuales se han de crear los objetos del sistema.

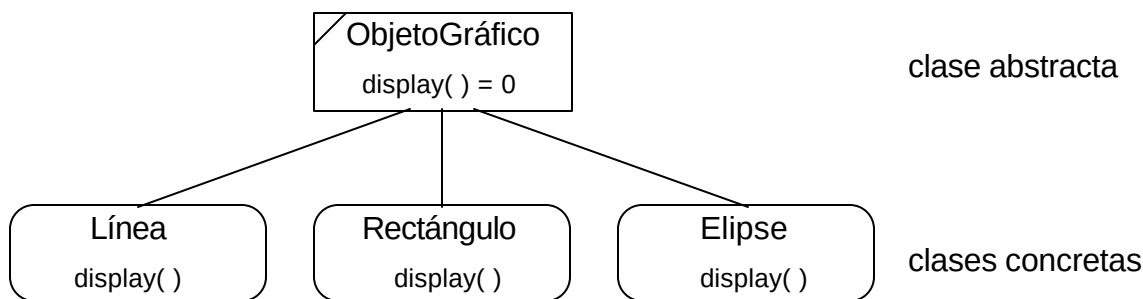


Fig. 12. Clases abstractas y clases concretas.

Volviendo al ejemplo de la jerarquía de objetos gráficos, la clase **ObjetoGráfico** es un ejemplo de clase abstracta que encapsula los atributos comunes (ancho de línea, color, estado, etc.) y especifica el comportamiento básico de todo objeto gráfico. Dentro de este comportamiento básico **debe** haber un método que pinte al objeto (**display**). Consecuentemente, este método es declarado en la clase **ObjetoGráfico** pero no definido (implementado). De esta manera se obliga a todas las clases descendientes a definirlo, cada una en su forma apropiada. Cuando un método

es declarado pero no definido se dice que es un *método abstracto* y toda clase que tenga al menos un método abstracto es una clase abstracta.

Con esto cubrimos los conceptos básicos involucrados en la metodología orientada por objetos. Ahora hacemos una breve presentación de la metodología de diseño que se seguirá en el curso y que está mayormente fundamentada en la metodología propuesta por Rebecca Wirfs-Brock, Brian Wilkerson y Lauren Wiener en [Wirfs-Brock].

Exploración inicial

En un principio, el proceso de diseño OO es exploratorio. Se buscan las clases que abstraen el conocimiento del sistema de la forma más natural y razonable. Los pasos a seguir son los siguientes:

1. Encontrar las clases del sistema.
2. Determinar el conocimiento y comportamiento de los cuales cada clase es responsable.
3. Determinar la forma en que los objetos colaboran entre sí para cumplir con sus responsabilidades.

Los pasos anteriores producen:

- una lista de clases,
- una descripción del conocimiento y de las operaciones de que cada es responsable, y
- una descripción de las colaboraciones entre las clases.

Análisis detallado

Con la información anterior se comienza la etapa analítica del proceso de diseño.

Primero se deben examinar las relaciones de herencia existentes entre las clases. Se analiza el comportamiento de cada clase, se buscan clases que compartan responsabilidades y se trata de factorizar estas responsabilidades en superclases.

Después se analizan las colaboraciones entre las clases con el fin de simplificarlas:

¿Hay partes del sistema donde el tráfico de mensajes es particularmente pesado?

¿Hay clases que colaboran con todas las demás clases?

¿Hay clases que no colaboran con ninguna otra clase?

¿Hay grupos de clases que colaboran entre sí en forma más cercana que con el resto?

Así la etapa analítica del proceso de diseño consiste de:

1. Factorización de responsabilidades con el fin de construir jerarquías de herencia.
2. Simplificación de las colaboraciones entre las clases.

Con esto es posible traducir las responsabilidades de cada clase en un conjunto de métodos completamente especificados (nombre, tipo y cantidad de parámetros, objeto de retorno) y así completar la lista de clases que se deben implementar.

Subsistemas

Dependiendo de la complejidad del sistema modelado, se pueden llegar a utilizar varios niveles de encapsulamiento, uno dentro de otro. Hasta ahora se ha hablado de las clases como las únicas entidades conceptuales que componen una aplicación, sin embargo, se pueden identificar grupos de clases que tienen cierta integridad lógica y que trabajan para cumplir con un conjunto determinado de responsabilidades relacionadas. Estos grupos de clases son llamados *subsistemas* y son entidades conceptuales que permiten trabajar a un nivel de abstracción mayor en el diseño de sistemas.

Las tres etapas anteriores, exploración inicial, análisis detallado e identificación de subsistemas, comprenden la metodología a seguir en el curso. En la siguiente sección analizamos las técnicas utilizadas en la identificación de las clases del sistema y comenzamos con el diseño de la aplicación que nos servirá de ejemplo durante el curso-taller.

3. Clases del sistema

La primera tarea en el diseño de un sistema es encontrar las clases que lo componen. Para esto comenzamos con la especificación de requerimientos del sistema¹. Esta debe incluir los objetivos de la aplicación, las entradas esperadas y las respuestas deseadas. Se debe leer cuidadosamente la especificación de requerimientos en busca de frases nominales. Se cambian plurales a singulares y se crea una lista preliminar. En esta lista aparecerán frases que incluyen clases obvias, frases que no tienen ningún sentido y frases sobre las cuales no se puede estar seguro. No todas las clases candidatas aparecerán en diseño final y algunas de las clases del sistema serán pasadas por alto en esta etapa. Se debe tener en cuenta que el lenguaje es muy flexible y que no existe la especificación de requerimientos completa. Además del sentido común se debe tener en cuenta entonces la siguiente guía:

- Se deben modelar los objetos físicos, tales como Mouse, Disco, Impresora, etc.
- Se deben modelar las entidades conceptuales que representen una abstracción coherente, tales como Ventana, Archivo, etc.
- Si aparece más de un nombre para un mismo concepto, se debe elegir el que sea más significativo en términos del resto del sistema.
- Se debe tener cuidado con los adjetivos. Un adjetivo puede ser utilizado para sugerir una clase diferente de objetos o para sugerir un uso diferente del mismo objeto. Si el uso de un adjetivo marca un comportamiento diferente, entonces se trata de una nueva clase.
- Se debe tener cuidado con las frases en voz pasiva. Las frases escritas en voz pasiva tienen implícito un sujeto. Se deben reescribir en voz activa y evaluar si el sujeto es una clase del sistema o no. De forma similar, hay que tener cuidado con las frases en voz activa donde el sujeto es algo que está por fuera del sistema, "el usuario", por ejemplo.
- Se deben modelar categorías de clases. Estas categorías serán superclases abstractas en el diseño final, sin embargo en esta primera etapa se deben modelar como clases específicas.
- Se deben modelar las interfaces con el mundo externo, tales como la interfaz de usuario, interfaz con otros programas o con el sistema operativo.
- En esta fase se debe modelar el valor de los atributos de los objetos, y no los atributos en sí.

¹ "La comprensión del dominio del problema es realmente el punto crucial del análisis de sistemas" [Coad].

El resultado de esta etapa es una lista preliminar de clases. Muchas clases faltarán y otras serán eliminadas después.

Especificación de requerimientos para un editor gráfico elemental.

Un editor gráfico es una aplicación gráfica interactiva. En esta aplicación el usuario puede crear y editar dibujos compuestos de líneas, rectángulos, elipses y textos. Estos dibujos pueden ser almacenados en disco y posteriormente recuperados como dibujo en edición (persistencia).

El usuario interactúa con el editor por medio de herramientas. Estas serán de dos tipos: herramientas de creación y herramientas de selección. Sólo una herramienta estará activa en determinado momento y ésta será seleccionada por menú o por interacción con el ícono correspondiente. Cada herramienta tendrá asociado un cursor en particular que la represente.

Cuando la herramienta de selección está activa los elementos del dibujo pueden ser seleccionados con el mouse. Se pueden seleccionar uno o más elementos. En este caso los elementos seleccionados pueden ser manipulados como si se tratara de un sólo elemento. La manipulación de elementos se reduce a cambio en los atributos de tamaño, posición, color y estado (activo/no activo). El grupo de elementos seleccionados debe mostrar sus puntos de control. Presionando el botón izquierdo del mouse sobre un punto de control y luego moviendo el mouse ocasiona el cambio de tamaño del elemento asociado. Presionando el botón dentro de la selección y luego moviendo el mouse ocasiona un cambio de posición.

Cuando la herramienta de creación está activa el grupo de elementos seleccionados esta vacío. Cada vez que un elemento es creado, la herramienta de selección es activada y el elemento pasa a ser el nuevo grupo seleccionado.

La herramienta de creación de texto cambia el cursor a una línea vertical proporcional al tamaño de letra en uso (I-beam). El texto insertado comienza en la posición donde el usuario oprime el botón izquierdo del mouse.

Las demás herramientas de creación comienzan la secuencia de eventos cuando el botón izquierdo es oprimido y terminan cuando el botón es liberado. En el caso de creación de líneas estos eventos representan el punto inicial y final. En los otros casos representan los puntos diagonales del área rectangular que contiene al elemento.

En el caso de las líneas, los puntos de control coinciden con sus puntos extremos. En los demás casos, los puntos de control coinciden con los vértices del rectángulo que contiene al elemento.

Lo que no dice

Se necesita una ventana donde crear los dibujos.

¿Qué sucede cuando ésta cambia de tamaño?

¿Cómo se cambia el color de un elemento?

¿Cómo se especifica el color de relleno de un elemento?

¿Cómo se especifica el color y tipo de texto en uso?

¿Cómo se borra un elemento?

¿Cómo se recupera?

¿Cómo se deshacen otras operaciones?

¿Se pueden imprimir los dibujos creados?

etc.

Especificación de requerimientos para un editor gráfico elemental.

Un editor gráfico es una aplicación gráfica interactiva. En esta aplicación el usuario puede crear y editar dibujos compuestos de líneas, rectángulos, elipses y textos. Estos dibujos pueden ser almacenados en disco y posteriormente recuperados como dibujo en edición (persistencia).

El usuario interactúa con el editor por medio de herramientas. Estas serán de dos tipos: herramientas de creación y herramientas de selección. Sólo una herramienta estará activa en determinado momento y ésta será seleccionada por menú o por interacción con el ícono correspondiente. Cada herramienta tendrá asociado un cursor en particular que la represente.

Cuando la herramienta de selección está activa los elementos del dibujo pueden ser seleccionados con el mouse. Se pueden seleccionar uno o más elementos. En este caso los elementos seleccionados pueden ser manipulados como si se tratara de un sólo elemento. La manipulación de elementos se reduce a cambio en los atributos de tamaño, posición, color y estado (activo/no activo). El grupo de elementos seleccionados debe mostrar sus puntos de control. Presionando el botón izquierdo del mouse sobre un punto de control y luego moviendo el mouse ocasiona el cambio de tamaño del elemento asociado. Presionando el botón dentro de la selección y luego moviendo el mouse ocasiona un cambio de posición.

Cuando la herramienta de creación está activa el grupo de elementos seleccionados esta vacío. Cada vez que un elemento es creado, la herramienta de selección es activada y el elemento pasa a ser el nuevo grupo seleccionado.

La herramienta de creación de texto cambia el cursor a una línea vertical proporcional al tamaño de letra en uso (I-beam). El texto insertado comienza en la posición donde el usuario oprime el botón izquierdo del mouse.

Las demás herramientas de creación comienzan la secuencia de eventos cuando el botón izquierdo es oprimido y terminan cuando el botón es liberado. En el caso de creación de líneas estos eventos representan el punto inicial y final. En los otros casos representan los puntos diagonales del área rectangular que contiene al elemento.

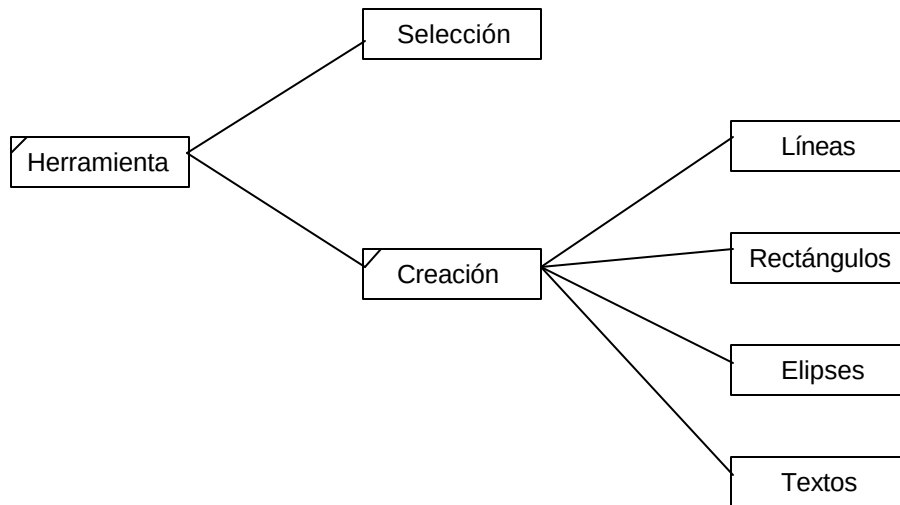
En el caso de las líneas, los puntos de control coinciden con sus puntos extremos. En los demás casos, los puntos de control coinciden con los vértices del rectángulo que contiene al elemento.

❶ Editor Gráfico: exploración inicial, lista preliminar de clases

### editor gráfico	aplicación
aplicación gráfica interactiva	idem anterior
### dibujo	
### línea	
### rectángulo	
### elipse	
### texto	
disco	ya modelado por el compilador: streams
usuario	aplicación sencilla: no se modela
### herramienta	
### herramientas de creación	(de líneas, de rectángulos, de elipses, de textos)
### herramientas de selección	una sola
### menú	(vista ya modelada, comportamiento no)
### ícono correspondiente (herramienta)	dónde?
elementos del dibujo	objetos gráficos: líneas, rectángulos, elipses, textos
### grupo de elementos seleccionados	
### puntos de control	
mouse	ya modelado por Windows
botón izquierdo del mouse	idem anterior
herramienta de selección	ya
### herramienta de creación de texto	
### evento	(undo/redo)
punto inicial y final	atributos, no clases
puntos diagonales	idem anterior
rectángulo que contiene al elemento	bounding box: atributo

Lo que dice la especificación de requerimientos:

Jerarquía de las herramientas:

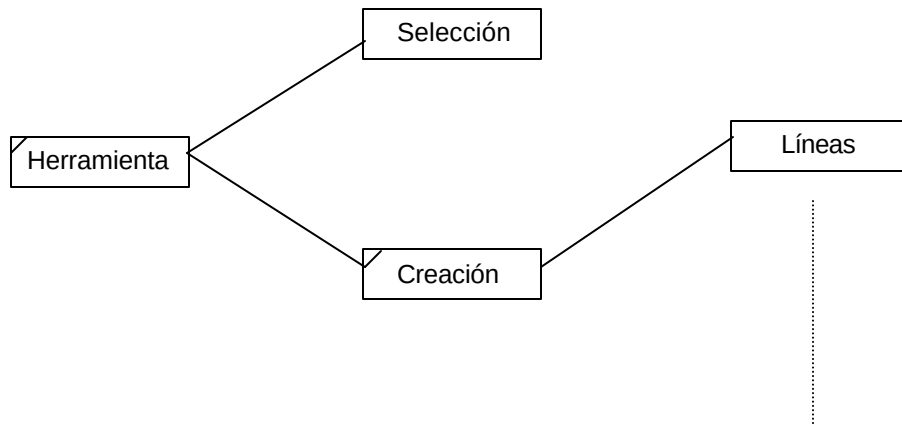


Lo que no dice la especificación de requerimientos:

¿dónde se pinta?	(ventana principal, ventana de edición)
¿íconos de herramientas, dónde?	(ventana flotante/speed bar)
¿cómo se cancela una operación?	(ESC)
¿cómo se cambia el color de un elemento?	(ventana de colores)
¿cómo se borra uno o más elementos?	(selección, DELETE)
¿undo/redo?	(eventos)
¿cómo se selecciona un grupo de elementos?	(SHIFT)
¿retícula de posicionamiento?	(snap on/off)
¿los elementos pueden ser rellenos?	(¿dónde se especifica/cambia?)
¿los elementos siempre tienen borde?	(¿dónde se especifica/cambia?)
¿impresión?	
¿ayuda en línea?	
¿que pasa con dibujos más grandes que la pantalla?	

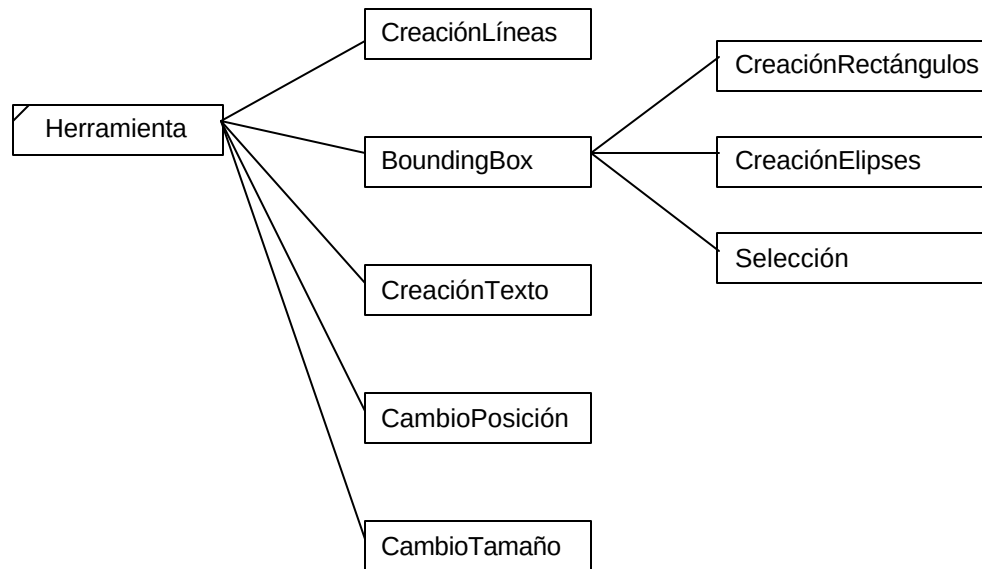
3.1 Clases concretas y clases abstractas

En esta lista preliminar ya se pueden diferenciar dos tipos de clases: clases que representan conceptos genéricos, tales como herramienta, elemento gráfico o evento, y clases que representan conjuntos de uno o más objetos concretos que han de conformar la aplicación: dibujo, herramienta para la creación de líneas, línea, elipse, etc.. La primera categoría está formada por las clases llamadas superclases *abstractas*. Estas clases sirven para encapsular el comportamiento y los atributos comunes a un grupo de clases y no se espera crear objetos a partir de ellas. Por el contrario, las clases del segundo grupo son llamadas *concretas* porque a partir de ellas se han construir los objetos de la aplicación. Muchas de estas clases concretas serán descendientes de las abstractas (elemento gráfico → línea, rectángulo, elipse, texto) y de ahí que estas últimas sean llamadas superclases. A nivel de diagrama es usual diferenciar las clases abstractas de las concretas por medio de una marca en la esquina superior-izquierda.

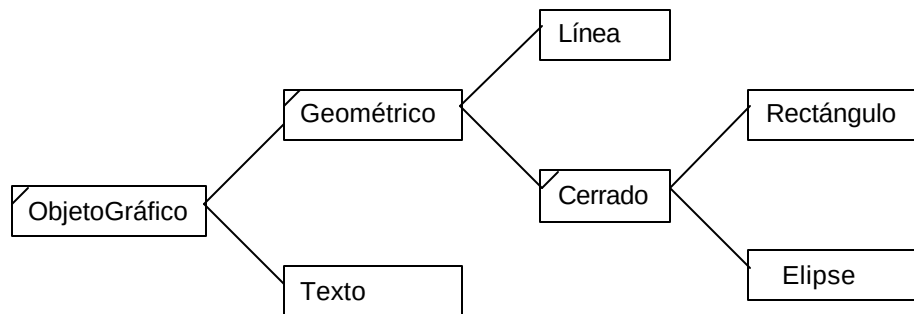


2 Editor Gráfico: exploración inicial, lista de clases

Herramientas, jerarquía de clases revisada:



Jerarquía de elementos gráficos

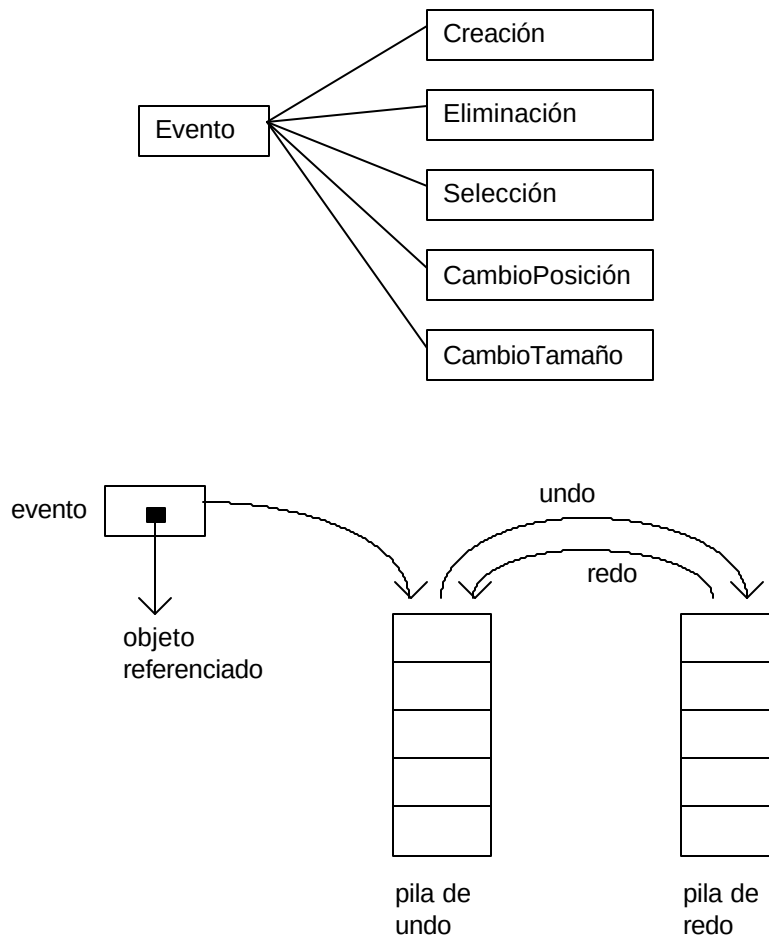


Nueva lista de clases:

0. Editor
1. Dibujo
2. ObjetoGráfico (superclase abstracta)
3. ObjetoGeométrico (superclase abstracta) (ancho de línea)
4. Línea
5. ObjetoGráficoCerrado (superclase abstracta) (borde/relleno)
6. Rectángulo
7. Elipse
8. Texto

9. Herramienta (superclase abstracta)
10. herramienta de CreaciónLíneas
11. herramienta BoundingBox
12. herramienta de CreaciónRectángulos
13. herramienta de CreaciónElipses
14. herramienta de Selección
15. herramienta de CreaciónTextos
16. herramienta de CambioPosición
17. herramienta de CambioTamaño
18. Menú (vista ya modelada, comportamiento no)
19. Icono correspondiente a cada herramienta
20. Grupo de elementos seleccionados
21. PuntoDeControl
22. Evento (superclase abstracta) (undo/redo)
23. Ventana principal
24. Ventana de edición
25. Ventana de herramientas
26. Ventana de ancho de línea
27. Ventana de paleta de colores

Jerarquía de eventos



mecanismo de undo/redo de múltiples niveles:

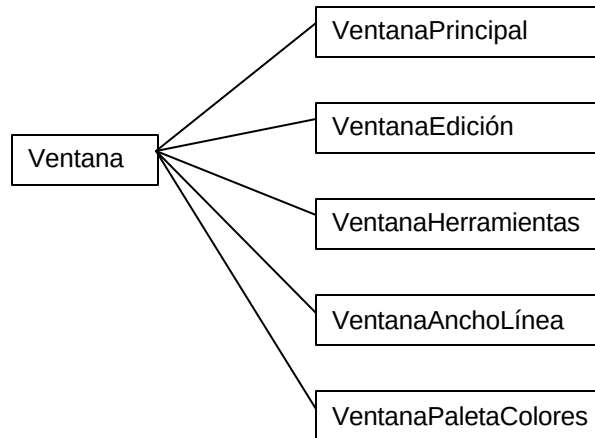
cada vez que el usuario genera un evento (relevante) relacionado con un elemento gráfico (creación, eliminación, selección, cambio de posición, cambio de tamaño), se crea y almacena en una pila (undo) un objeto evento que contiene una referencia (apuntador) al elemento gráfico en cuestión. Este objeto evento 'sabe' como deshacer/rehacer la operación realizada sobre el elemento gráfico referenciado. Para deshacer una operación se saca el último elemento agregado a la pila (LIFO) de 'undo', se le pide que deshaga la operación asociada y se agrega a la pila de 'redo'. Para rehacer una operación se saca el último elemento agregado a la pila de 'redo', se le pide que rehaga la operación asociada y se agrega a la pila de 'undo'.

Nueva lista de clases:

0. Editor
1. Dibujo
2. ObjetoGráfico (superclase abstracta)
3. ObjetoGeométrico (superclase abstracta)
4. Línea
5. ObjetoCerrado (superclase abstracta)
6. Rectángulo
7. Elipse
8. Texto
9. Herramienta (superclase abstracta)
10. herramienta de CreaciónLíneas
11. herramienta BoundBox
12. herramienta de CreaciónRectángulos
13. herramienta de CreaciónElipses
14. herramienta de Selección
15. herramienta de CreaciónTextos
16. herramienta de CambioPosición
17. herramienta de CambioTamaño
18. Menú (vista ya modelada, comportamiento no)
19. Icono correspondiente a cada herramienta
20. Grupo de elementos seleccionados
21. PuntoDeControl
22. Evento (superclase abstracta)
23. evento de Creación
24. evento de Eliminación
25. evento de Selección
26. evento de CambioPosición
27. evento de CambioTamaño
28. Ventana principal
29. Ventana de edición
30. Ventana de herramientas
31. Ventana de ancho de línea
32. Ventana paleta de colores

④ Editor Gráfico: exploración inicial, lista de clases

Jerarquía de ventanas



Lista preliminar de clases:

0. Editor
1. Dibujo
2. ObjetoGráfico (superclase abstracta)
3. ObjetoGeométrico (superclase abstracta)
4. Línea
5. ObjetoCerrado (superclase abstracta)
6. Rectángulo
7. Elipse
8. Texto
9. Herramienta (superclase abstracta)
10. herramienta de CreaciónLíneas
11. herramienta BoundingBox
12. herramienta de CreaciónRectángulos
13. herramienta de CreaciónElipses
14. herramienta de Selección
15. herramienta de CreaciónTextos
16. herramienta de CambioPosición
17. herramienta de CambioTamaño
18. Menú (Borland)
19. Icono correspondiente a cada herramienta
20. Grupo de elementos seleccionados
21. PuntoDeControl
22. Evento (superclase abstracta)
23. evento de Creación
24. evento de Eliminación
25. evento de Selección
26. evento de CambioPosición

- 27. evento de CambioTamaño
- 28. Ventana (superclase: Borland)
- 29. Ventana principal (Borland)
- 30. Ventana de edición
- 31. Ventana de herramientas
- 32. Ventana de ancho de línea (Borland)
- 33. Ventana paleta de colores

Nota: el programa no provee soporte de clipboard (cut/copy/paste), se deja para otra versión, con el fin de estudiar el impacto producido por cambios en la especificación de requerimientos.

Nota: el programa no provee soporte para impresión, idem anterior.

Nota: el programa no provee soporte para dibujos más grandes que el área de la ventana de edición (scroll bars), idem anterior.

Nota: el programa no provee soporte para zoom, idem anterior.

Nota: el programa no provee soporte para snap-on, idem anterior.

Nota: el programa no provee soporte para impresión, idem anterior.

Nota: el programa no provee soporte para MDI, idem anterior.

3.2 Registro de clases

Cada clase identificada es registrada en una tarjeta donde se anota su nombre, si es una clase concreta o abstracta. Al respaldo se anota en forma breve el propósito de la clase dentro de la aplicación. Estas 'tarjetas' pueden ser del tamaño de una tarjeta de índice, una hoja tamaño carta o un documento en computador. La ventaja de utilizar tarjetas de papel está en que se pueden reorganizar fácilmente y tener una visión global de las clases de la aplicación en todo momento. Estas mismas tarjetas servirán más adelante para registrar las responsabilidades de cada clase.

Clase: ObjetoGráfico		(@)

Propósito:
Encapsular comportamiento y atributos comunes a todos los los elementos gráficos que la aplicación maneja.

4. Responsabilidades

En la sección anterior se identificaron las clases de la aplicación. En esta sección se define el propósito de las clases asignandoles conocimiento y comportamiento, esto es, se determinan las responsabilidades de cada clase.

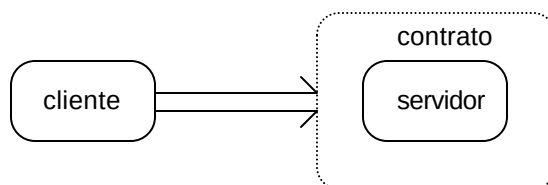
Las responsabilidades incluyen dos tópicos:

1. el conocimiento que la clase debe mantener
2. las acciones que la clase debe realizar

Las responsabilidades de una clase, y por lo tanto de todo objeto de esa clase, están representadas en los servicios¹ que la clase provee para los contratos que soporta.

4.1 Clientes y Servidores

Las colaboraciones entre objetos son interacciones en un solo sentido: un objeto solicita un servicio a otro objeto. El objeto que solicita el servicio es llamado *cliente* y el objeto que provee el servicio es llamado *servidor*.



La formas válidas en las cuales un objeto cliente puede interactuar con un objeto servidor están especificadas en un *contrato*. De esta manera, en un contrato aparecen todos los servicios que un objeto cliente puede solicitar a un objeto servidor. Ambos objetos deben cumplir con el contrato: el objeto cliente solo solicita los servicios especificados y el objeto servidor responde en una forma apropiada. Aquí se debe tener en cuenta que *cliente* y *servidor* son roles. Un rol no es atributo inherente a una clase de objetos, sino que un objeto puede asumir cualquiera de los roles en diferentes momentos. Por lo tanto, un objeto puede participar en diferentes contratos y es *responsable* de responder por éstos. Los conceptos de cliente y servidor sirven para llevar un control de las relaciones entre objetos en tiempos específicos.

La especificación formal de un contrato estará representada por un conjunto de uno o más métodos de la interfaz pública del objeto servidor. Por ahora se especifica lo **qué** deben hacer los métodos y no **cómo**.

¹ Un servicio puede tratarse de una acción a realizar o de cierta información a retornar.

4.2 Identificación de responsabilidades

Las responsabilidades pueden ser identificadas de varias formas. Por el momento las fuentes de información son la especificación de requerimientos y la lista preliminar de clases.

Se debe leer nuevamente la especificación de requerimientos, pero esta vez en busca de **verbos** que claramente (sentido común) representen acciones que algún objeto del sistema deba llevar a cabo. De la misma forma, cada vez que se mencione algún tipo de información se debe anotar. La información manipulada por el sistema también representa responsabilidades de algún objeto del mismo.

La lista preliminar de clases es otra fuente de información. El propósito anotado en cada tarjeta de clase es índice de nuevas responsabilidades. ¿Qué conocimiento y acciones están implicadas en el propósito? ¿Qué responsabilidades tiene la clase respecto al manejo de sus atributos?

Especificación de requerimientos para un editor gráfico elemental.

Un editor gráfico es una aplicación gráfica interactiva. En esta aplicación el usuario puede crear y editar dibujos compuestos de líneas, rectángulos, elipses y textos. Estos dibujos pueden ser almacenados en disco y posteriormente recuperados como dibujo en edición (persistencia).

El usuario interactúa con el editor por medio de herramientas. Estas serán de dos tipos: herramientas de creación y herramientas de selección. Sólo una herramienta estará activa en determinado momento y ésta será seleccionada por menú o por interacción con el ícono correspondiente. Cada herramienta tendrá asociado un cursor en particular que la represente.

Cuando la herramienta de selección está activa los elementos del dibujo pueden ser seleccionados con el cursor. Se pueden seleccionar uno o más elementos. En este caso los elementos seleccionados pueden ser manipulados como si se tratara de un sólo elemento. La manipulación de elementos se reduce a cambio en los atributos de tamaño, posición, color y estado (activo/no activo). El grupo de elementos seleccionados debe mostrar sus puntos de control. Presionando el botón izquierdo del mouse sobre un punto de control y luego moviendo el mouse ocasiona el cambio de tamaño del elemento asociado. Presionando el botón dentro de la selección y luego moviendo el mouse ocasiona un cambio de posición.

Cuando la herramienta de creación está activa el grupo de elementos seleccionados esta vacío. Cada vez que un elemento es creado, la herramienta de selección es activada y el elemento pasa a ser el nuevo grupo seleccionado.

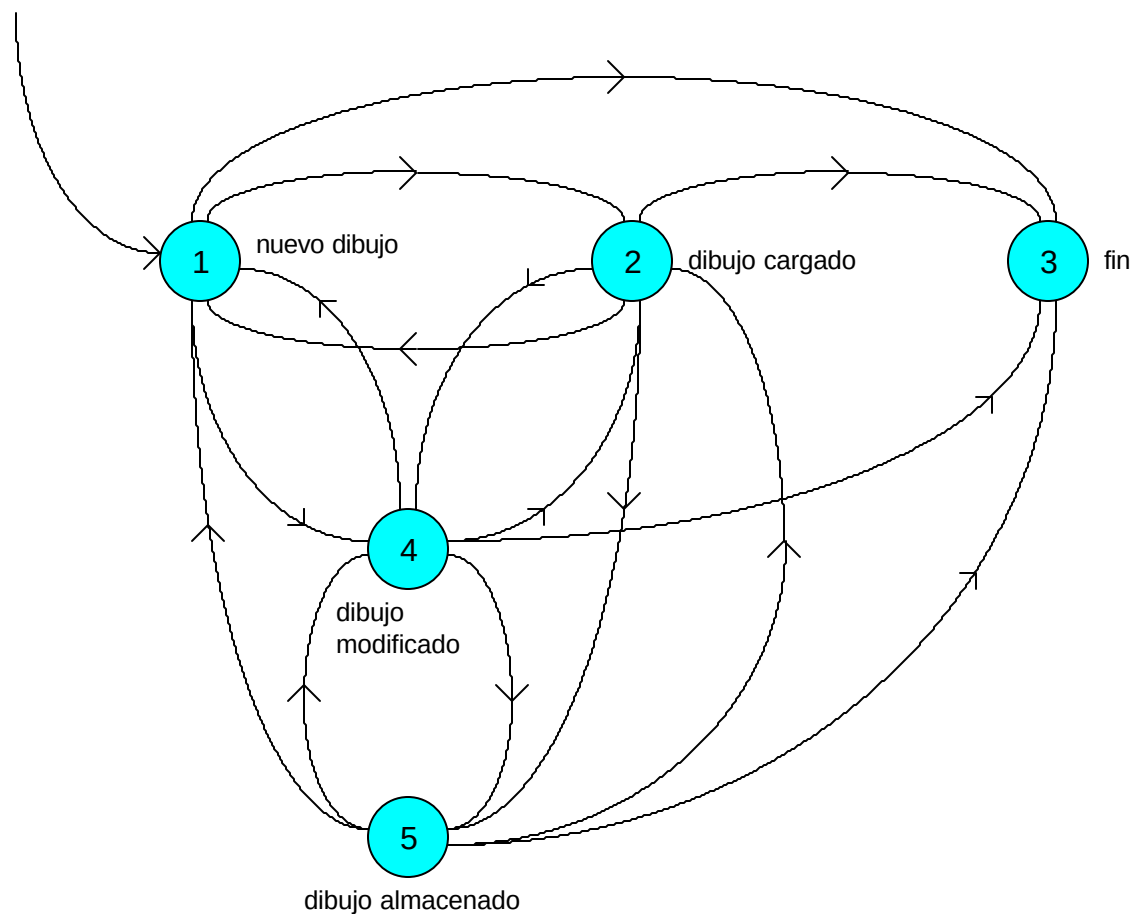
La herramienta de creación de texto cambia el cursor a una línea vertical proporcional al tamaño de letra en uso (I-beam). El texto insertado comienza en la posición donde el usuario oprime el botón izquierdo del mouse.

Las demás herramientas de creación comienzan la secuencia de eventos cuando el botón izquierdo es oprimido y terminan cuando el botón es liberado. En el caso de creación de líneas estos eventos representan el punto inicial y final. En los otros casos representan los puntos diagonales del área rectangular que contiene al elemento.

En el caso de las líneas, los puntos de control coinciden con sus puntos extremos. En los demás casos, los puntos de control coinciden con los vértices del rectángulo que contiene al elemento.

Editor Gráfico

Diagrama de Transición de Estados



- Universidad Nacional de Colombia. Facultad de Ingeniería. Departamento de Ingeniería de Sistemas.
Unidad de Educación Continuada.

4.2 { elemento/s borrado/s
 4.3 { elemento/s seleccionado/s
 4.4 { cambio de atributos elemento { color
 { color relleno
 { ancho de línea
 { borde/no
 { relleno/no
 { posición
 { tamaño
 { texto
 { fuente

5. dibujo almacenado

###1 Crear objetos principales e inicializar valores por omisión:

- crear aplicación (función main)
- crear ventana principal
- crear ventana de edición
- crear speed bar de anchos de línea
- crear ventana de colores
- crear herramientas
- crear ventana de herramientas
- crear contenedor de objetos gráficos (Dibujo)
- crear contenedor de objetos seleccionados (Grupo)
- crear menú
- asociar aceleradores
- iniciar color de línea
- iniciar ancho de línea
- iniciar color de fondo
- iniciar fuente por omisión
- iniciar flag borde (TRUE)
- iniciar flag relleno (FALSE)
- iniciar flag mouse_down (FALSE)
- iniciar flag dibujo_modificado (FALSE)
- iniciar nombre dibujo (<sin nombre>)
- iniciar título ventana principal (Editor Gráfico - <sin nombre>)
- iniciar flag herramienta activa (creación de líneas)
- cargar cursor herramienta activa
- inhabilitar entrada del menú 'almacenar'
- inhabilitar entrada del menú 'almacenar como'

1###2 Cargar un dibujo

- leer nombre de archivo
- cargar dibujo
- actualizar nombre dibujo
- actualizar título ventana principal (Editor Gráfico - nombre dibujo)
- repintar

1###3 Terminar después de haber iniciado un nuevo dibujo

- destruir objetos creados en ###1

1###4.1 Crear un nuevo elemento

- crear objeto
- agregarlo al dibujo
- flag dibujo_modificado = TRUE
- repintar

2###1 Comenzar nuevo dibujo después de haber cargado uno (no modificado)

- flush de Dibujo
- actualizar nombre dibujo (<sin nombre>)
- actualizar título ventana principal
- repintar

2###3 Terminar después de haber cargado un dibujo

- destruir objetos cargados en el dibujo
- destruir objetos creados en ###1

2###4.1 Crear un nuevo elemento

- idem 1###4.1

2###4.2 Borrar los elementos seleccionados

- marcar los elementos referenciados en el Grupo como no-activos
- flush del Grupo
- inhabilitar entrada 'borrar' del menú
- flag dibujo_modificado = TRUE
- repintar

2###4.3 Seleccionar elemento/s

- marcar los elementos seleccionados
- agregar los elementos seleccionados al grupo

2###4.4 Cambiar atributos elemento seleccionado

- { color
 - { color relleno
 - leer evento LBUTTONDOWN/RBUTTONDOWN en paleta de colores
 - actualizar color/color relleno del objeto
 - repintar
- { ancho de línea
 - leer evento LBUTTONDOWN en speed-bar anchos de línea
 - actualizar ancho de línea del objeto
 - repintar
- { borde/no
 - { relleno/no
 - actualizar estado del objeto
 - repintar
- { posición
 - verificar evento LBUTTONDOWN sobre algún elemento seleccionado
 - procesar eventos MOUSEMOVE (mover grupo)
 - procesar evento LBUTTONUP
- { tamaño
 - verificar que el grupo sólo tiene un elemento seleccionado
 - verificar evento LBUTTONDOWN sobre algún punto de control del elemento seleccionado
 - procesar eventos MOUSEMOVE (cambiar tamaño del elemento)
 - procesar evento LBUTTONUP
- { texto
 - { fuente
 - procesar evento LBUTTONDOWNBLCLK (abrir caja de diálogo)
 - procesar eventos en caja de diálogo
 - actualizar atributos del elemento de texto si el usuario cierra la caja de diálogo con OK
 - repintar

2###5 Almacenar dibujo cargado (no modificado)

- almacenar dibujo

4###1 Iniciar nuevo dibujo después de haber modificado el dibujo en edición

- preguntar al usuario si desea almacenar el dibujo
 - almacenar dibujo
- destruir objetos en el dibujo
- actualizar nombre dibujo (<sin nombre>)
- actualizar título ventana principal (Editor Gráfico - <sin nombre>)
- repintar

4###2 Cargar dibujo después de haber modificado el dibujo en edición

- preguntar al usuario si desea almacenar el dibujo
 - almacenar dibujo
- destruir objetos en el dibujo
- idem 1###2

4###3 Terminar después de haber modificado el dibujo en edición

- preguntar al usuario si desea almacenar el dibujo
 - almacenar dibujo
- destruir objetos en el dibujo
- destruir objetos creados en ###1

4###5 Almacenar dibujo en edición

- almacenar dibujo
- flag dibujo_modificado = FALSE

5###1 Iniciar nuevo dibujo después de haber almacenado el dibujo en edición

- destruir objetos en el dibujo
- actualizar nombre dibujo (<sin nombre>)
- repintar

5###2 Cargar dibujo después de haber almacenado el dibujo en edición

- destruir objetos en el dibujo
- idem 1###2

5###3 Terminar después de haber almacenado el dibujo en edición

- idem 2###3

5###4.1 Crear un nuevo elemento

- idem 2###4.1

5###4.2 Borrar los elementos seleccionados

- idem 2###4.2

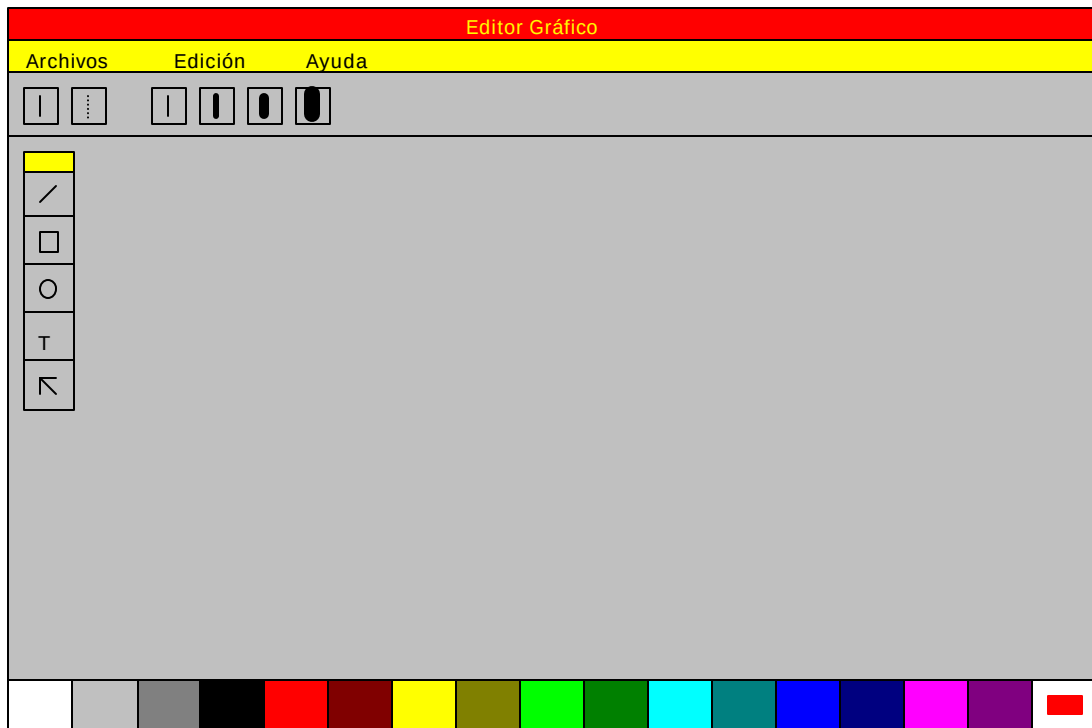
5###4.3 Seleccionar elemento/s

- idem 2###4.3

5###4.4 Cambiar atributos elemento seleccionado

- idem 2###4.4
-

Otra fuente de responsabilidades es el 'layout' de la aplicación que incluye la ventana principal, ventanas auxiliares, y entradas de menú. Analizando las entradas de menú obtenemos nuevas responsabilidades que en primera instancia son asignadas a la ventana principal, pero que ésta seguramente delegará en otras clases de la aplicación.



4.3 Asignación de responsabilidades

En principio no existe otra regla que el sentido común para la asignación de las responsabilidades identificadas. Muchas de las responsabilidades identificadas están asociadas en forma evidente a determinadas clases de la aplicación. Es claro que es responsabilidad de la ventana principal responder a los comandos de menú, así como mantener información sobre el estado del dibujo (si ha sido modificado o no, por ejemplo). Es claro que es responsabilidad de cada objeto gráfico el saber como pintarse. Sin embargo aparecen responsabilidades que no son atribuibles en forma clara a ninguna de las clases identificadas, o que por el contrario podrían ser atribuidas a más de una. ¿Quién es responsable de mantener información sobre la herramienta activa? ¿La ventana de edición o la ventana de herramientas? ¿O la misma herramienta?. El primer caso es índice de que falta identificar una o más clases en el sistema. El segundo caso implica, por lo general, una decisión de diseño.

A continuación se presentan algunas guías para la asignación de responsabilidades:

- i. distribuir la inteligencia del sistema en forma equilibrada
 - ii. especificar las responsabilidades en la forma más general posible
 - iii. mantener juntos comportamiento e información relacionada
 - iv. mantener información sobre cada tópico en un solo lugar
 - v. distribuir las responsabilidades complejas entre varias clases
- i. Se puede decir que todo sistema tiene cierta 'inteligencia', representada en lo que 'sabe' (la información que mantiene), las acciones que 'sabe' realizar y la forma en que interactúa con otros sistemas, incluidos los usuarios. Esta inteligencia estará conformada por la inteligencia de las clases dentro del sistema, algunas muy inteligentes, otras menos. La inteligencia de una clase depende a su vez de la información que mantiene, las acciones que puede realizar y el impacto que tiene sobre otras clases del sistema. Un contenedor, una lista por ejemplo, es un objeto que agrupa otros objetos y tiene poco impacto sobre el resto de la aplicación, inclusive sobre los objetos que contiene: se consideran objetos poco inteligentes. Los objetos gráficos, como los que contiene el editor que estamos diseñando, son objetos un poco más inteligentes: por lo menos saben como interactuar con el sistema operativo para pintarse a sí mismos. Seguramente también saben como cambiar de posición, de tamaño o color. Un objeto herramienta ya es un objeto bastante inteligente ya sabe como interactuar con el usuario y como crear objetos gráficos. Cuando se diseñan las clases del sistema, se debe distribuir la inteligencia del mismo entre estas clases y para esto se pueden seguir dos enfoques opuestos: minimizar la cantidad de clases inteligentes, o, distribuir la inteligencia del sistema en forma equilibrada. El primer enfoque tiene la ventaja de permitir una comprensión más clara e inmediata del flujo de control de la aplicación. Sin embargo tiene la desventaja de generar un sistema centralizado, poco flexible frente a modificaciones futuras. El segundo enfoque promueve idealmente la generación de clases igualmente inteligentes, un enfoque puramente orientado por objetos, pero también poco realizable. Con este enfoque se generan aplicaciones más flexibles y más fáciles de modificar o

extender. Se trata entonces de distribuir la inteligencia del sistema en la forma más equilibrada posible, reconociendo que van a haber clases más inteligentes que otras.

ii. Especificar las responsabilidades en la forma más general posible. Se podría decir que es responsabilidad de la clase Línea el saber pintar una línea, o de la clase Rectángulo el saber pintar un rectángulo. Sin embargo es más conveniente especificar que todo objeto gráfico debe saber como pinarse a sí mismo. Si se especifican las responsabilidades en forma general, se encontrarán clases que comparten responsabilidades y de esta forma se descubrirán más fácilmente las relaciones de herencia.

iii. Mantener juntos comportamiento e información relacionada. Si una clase es responsable de mantener cierta información, es lógico entonces que también sea responsable de las operaciones necesarias sobre esa información. De la misma forma, si una clase requiere cierta información para realizar una tarea, es lógico asignarle la responsabilidad de mantener esa información. De esta manera se reduce el tráfico de mensajes entre los objetos de la aplicación.

iv. Mantener información sobre cada tópico en un solo lugar. Por lo general, la responsabilidad de mantener una información específica no debería ser compartida. El compartir información implica duplicidad, y la duplicidad puede llevar a la inconsistencia. En el diseño de ejemplo, la clase Dibujo es responsable de conocer los objetos gráficos que debe pintar, y cada ObjetoGráfico es responsable de saber la posición donde debe pintarse. Si la posición de los objetos gráficos en el dibujo fuera mantenida también en la clase Dibujo, cada vez que un ObjetoGráfico cambiara de posición debería enviar un mensaje a la clase Dibujo para mantener la consistencia de la información.

v. Distribuir las responsabilidades complejas entre varias clases. Muchas veces se encuentra que una responsabilidad representa varias responsabilidades en diferentes clases. En el editor gráfico, cuando la ventana principal queda al descubierto (se cierra otra aplicación que la cubría, por ejemplo), recibe un mensaje de Windows que le indica que debe repintarse y repintar su contenido (WM_PAINT). La ventana principal pinta su marco, título, menú y demás elementos (así cumple con su parte de la responsabilidad), y luego envía un mensaje al Dibujo para que este se pinte. La clase Dibujo cumple con su parte recorriendo la lista de objetos gráficos y enviándole a cada uno el mensaje 'píntese'. Cada objeto gráfico se pinta de acuerdo a sus atributos (color, posición, etc.), cumpliendo así con su parte de la responsabilidad original.

4.4 Registro de las responsabilidades

A medida que se van identificando y asignando las responsabilidades del sistema, se van registrando en las tarjetas creadas para el registro de las clases. Si las responsabilidades de una clase no caben en la tarjeta correspondiente, esto puede ser índice de que la clase centraliza la inteligencia del sistema y que se necesita un análisis más profundo que permita redistribuir las responsabilidades identificadas.

Clase: Dibujo		(C)
Atributos: Lista de elementos Lista de elementos seleccionados		
Responsabilidades		
Agrupar los elementos del dibujo		
Agrupar los elementos seleccionados		
Pintar el dibujo		
Almacenar en disco los elementos del dibujo		

5. Colaboraciones

Las clases cumplen con sus responsabilidades de dos maneras: llevando a cabo las tareas necesarias por si mismas o con la colaboración de otras clases. En las secciones anteriores se presentaron los lineamientos para identificar las clases que componen una aplicación y algunas guías para la asignación de responsabilidades. Ahora se debe determinar la forma en que las clases interactúan para cumplir con sus responsabilidades.

Una colaboración está representada por un servicio que el objeto cliente solicita al objeto servidor, con el fin de cumplir con una de sus responsabilidades. El patrón de colaboraciones entre las clases es importante dado que revela el flujo de control e información de la aplicación en tiempo de ejecución. A medida que se van identificando colaboraciones, se descubren los caminos de comunicación entre las clases y estos caminos permitirán en definitiva identificar subsistemas de clases. Los subsistemas nos permitirán a su vez encapsular aún más el conocimiento y comportamiento del sistema.

5.1 Identificación de colaboraciones

Se comienza examinando las responsabilidades de cada clase en busca de dependencias. Si una clase no tiene el conocimiento suficiente para llevar a cabo una actividad, entonces necesita de otra que sí lo tenga. Para cada responsabilidad de cada clase se deben hacer las siguientes preguntas:

- ¿Puede la clase cumplir con esta responsabilidad por si sola?
- Si no puede, ¿qué le hace falta?
- ¿De qué clases puede obtener lo que necesita?

De esta forma, toda responsabilidad que una clase no pueda cumplir por si sola representa una colaboración. Aquí es bueno recalcar que una colaboración es una interacción en un solo sentido. Algunas clases serán clientes, otras servidoras, y otras asumirán el rol de cliente en determinados momentos y el rol de servidoras en otros¹. Clases que representan interfaces con el mundo externo, por ejemplo, son clases servidoras. Como veremos, la clase Dibujo asume ambos roles. La clase VentanaPrincipal es un ejemplo de una clase mayormente cliente.

El análisis de otras relaciones entre las clases resulta útil también en la identificación de colaboraciones: relaciones "es parte de", relaciones "tiene conocimiento de" y relaciones "depende de". No todas estas relaciones serán relevantes en todos los diseños, y para algunos diseños será necesario analizar otras relaciones específicas para la aplicación. Sin embargo representan una buena guía para la identificación de nuevas colaboraciones.

¹ Una clase que no colabora con nadie y que no requiere colaboración alguna, debería ser descartada.

La relación todo-parte aparece casi siempre indicada en la especificación de requerimientos con frases como "Los dibujos están compuestos por elementos gráficos", esto es, los elementos gráficos son parte del dibujo. Las relaciones todo-parte implican por lo general la responsabilidad (del todo) de mantener cierta información. La responsabilidad principal de la clase Dibujo es llevar el control de los ObjetosGráficos que el usuario ha creado. En este caso, Dibujo es un ejemplo de una clase *compuesta*. Las relaciones todo-parte pueden ser a su vez de dos tipos: relaciones entre clases compuestas y los objetos que la componen, y relaciones entre contenedores (vectores, listas, ...) y los elementos que contienen. Dibujo es un ejemplo de una clase compuesta ya que tiene como responsabilidad no sólo agrupar los elementos gráficos, sino también bajarlos a disco, cargarlos de disco, e indicarles que deben pintarse cada vez que el dibujo debe ser repintado. Cada elemento gráfico es responsable de saber pintarse, Dibujo cumple con una responsabilidad mayor que es saber cuándo y cuáles elementos deben pintarse. Esta relación todo-parte entre Dibujo y ObjetosGráficos lleva implícita la colaboración que ObjetosGráfico le brinda a Dibujo para que éste cumpla con una de sus responsabilidades.

Las relaciones entre contenedores y elementos pueden o no implicar colaboraciones, y esto deberá ser analizado en el contexto de cada caso. Por lo general, el contenedor no necesita enviar mensajes a los objetos que contiene. En un vector por ejemplo, los objetos son accedidos por medio de un índice, por otros objetos de la aplicación. De hecho, en el ejemplo del editor gráfico que nos interesa, Dibujo *utiliza* un vector como estructura en memoria para almacenar los ObjetosGráficos. Otros contenedores como una lista ordenada pueden requerir que un objeto se compare con los otros de la lista antes de ser agregado.

Algunas clases tienen conocimiento de otras sin estar compuestas por estas últimas. Esta relación implica una responsabilidad de saber cierta información, y por lo tanto, una colaboración. Por ejemplo, todo ObjetoGráfico tiene asociados cuatro o más PuntosDeControl. Los ObjetosGráficos no están compuestos por PuntosDeControl, ni vice-versa, sin embargo cada PuntoDeControl debe conocer el ObjetoGráfico a cual está asociado para poder modificarlo. Visto de otra forma, un ObjetoGráfico requiere la colaboración de sus PuntosDeControl para cambiar de forma o tamaño.

Como ejemplo de relación "depende de" tenemos que los objetos gráficos en el dibujo se encuentran por debajo o por encima de otro. En este caso puede ser responsabilidad de cada ObjetoGráfico saber que objeto tiene por encima y cual por debajo, o puede ser responsabilidad de otra clase (como Dibujo) el conocer este orden.

5.2 Registro de colaboraciones

Las colaboraciones no existen sino en el ámbito de cumplir con una responsabilidad y por lo tanto siempre están asociadas con una de éstas. En las tarjetas de las clases, a la derecha de cada responsabilidad se anota el nombre de la clase servidora que colabora en el cumplimiento de esa responsabilidad.

Clase: Dibujo		(C)
Atributos: Lista de elementos Lista de elementos seleccionados (Grupo)		
Responsabilidades		
Agrupar los elementos del dibujo		Lista
Agrupar los elementos seleccionados		Grupo
Pintar el dibujo		ObjetoGrafico
Almacenar en disco los elementos del dibujo		ObjetoGrafico

Con la identificación de colaboraciones terminamos la etapa exploratoria, o de 'diseño preliminar', según Rebecca Wirfs-Brock et al. en [Wirfs-Brock]. Coad y Yourdon [Coad], sin embargo, llamarían a esto análisis orientado por objetos. Esto último parece más apropiado ya que realmente lo que se ha hecho es un análisis (identificación) de las clases que componen el sistema y sus roles dentro de éste. Los resultados de la fase exploratoria son :

1. Una lista preliminar de las clases que conforman la aplicación;
2. Una descripción del conocimiento y operaciones de los cuales cada clase es responsable;
3. Una descripción del esquema de colaboración entre éstas.

Todavía no se tiene un diseño, en la medida que no se tiene una especificación completa y detallada de las clases. En la siguiente etapa, llamada 'fase analítica' en esta metodología, se construirá un diseño final directamente implementable, aprovechando los resultados anteriores y otros beneficios de la metodología orientada por objetos.

6. Grafos de jerarquía

Con la información obtenida en la primera etapa se propone comenzar la fase analítica del diseño. El fin de este análisis está en obtener un nivel de comprensión más global del sistema que el obtenido en la fase exploratoria. Para esto se cuenta con las siguientes herramientas conceptuales y gráficas:

- ### grafos de jerarquías,
- ### diagramas de Venn, y
- ### contratos entre las clases

Se pretende estructurar las jerarquías de herencia para aprovechar los beneficios de la tecnología OO. Las clases candidatas son reexaminadas buscando grupos que compartan uno o más atributos útiles (aquellos que implican un comportamiento común de las clases que lo contienen): de cada uno de estos grupos surgirá una clase abstracta que será superclase de otras clases concretas, y eventualmente raíz de una jerarquía.

En general, las clases abstractas representan categorías de clases y se diseñan con el solo propósito de heredar de ellas: no van a existir instanciaciones de estas clases.

Las clases concretas son diseñadas en primer lugar para ser instanciadas, y eventualmente como superclases de las cuales heredar.

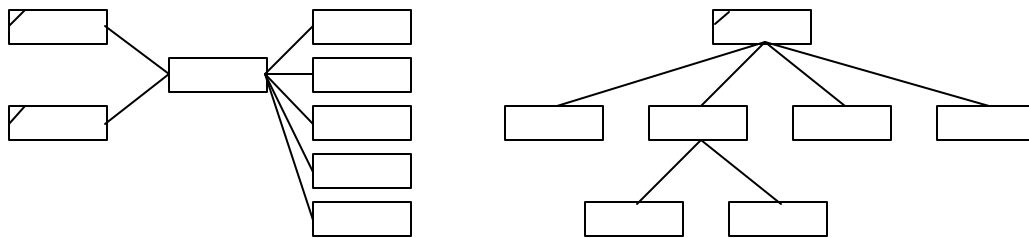


Fig. 1. Grafos de jerarquías de clases

En esta etapa se reexamina el comportamiento de cada clase y las relaciones de herencia con las otras clases del sistema, tratando de factorizar responsabilidades comunes que serán reubicadas en las superclases. Si miramos las clases como conjuntos de responsabilidades, otra herramienta que ayuda a obtener una comprensión más global del sistema está en los diagramas de Venn. Por cada jerarquía de clases se puede construir un diagrama de Venn equivalente.

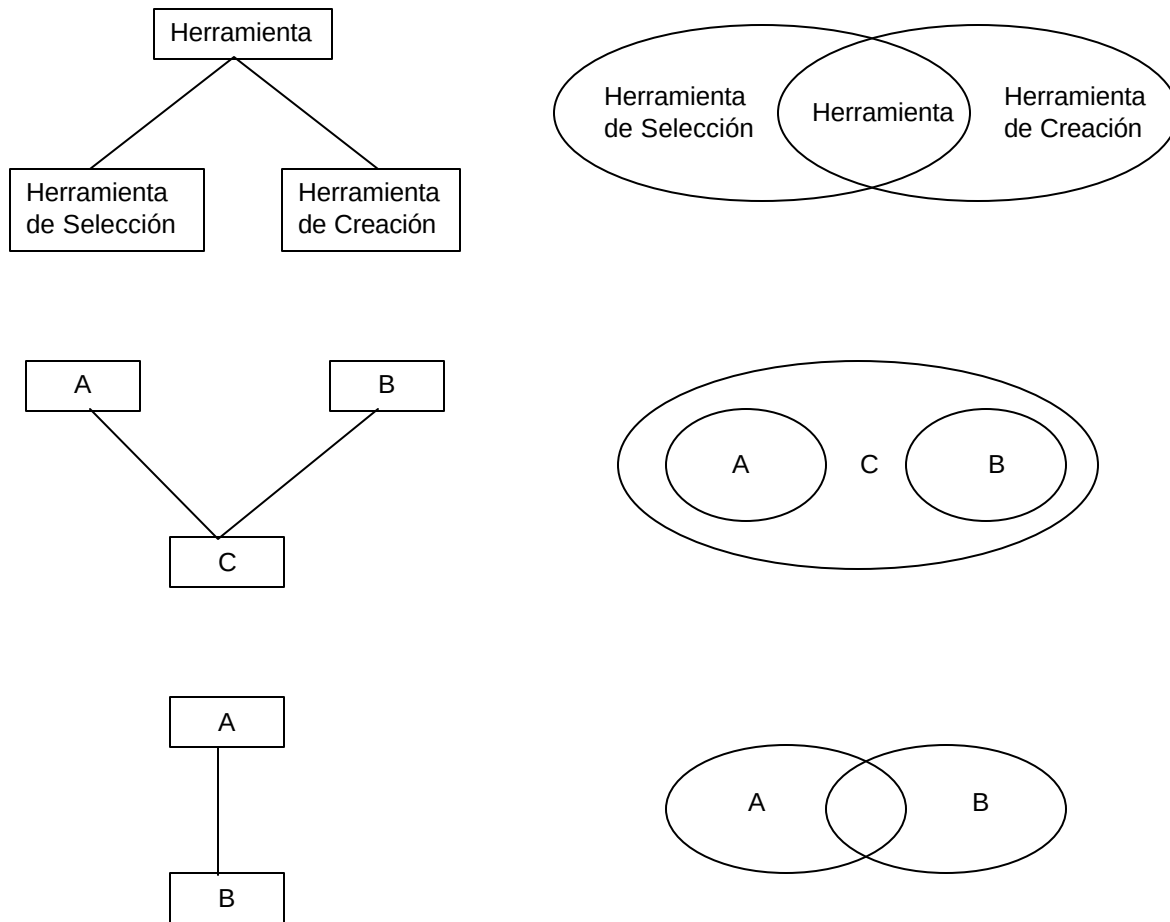


Fig. 2. Ejemplos de diagramas de Venn

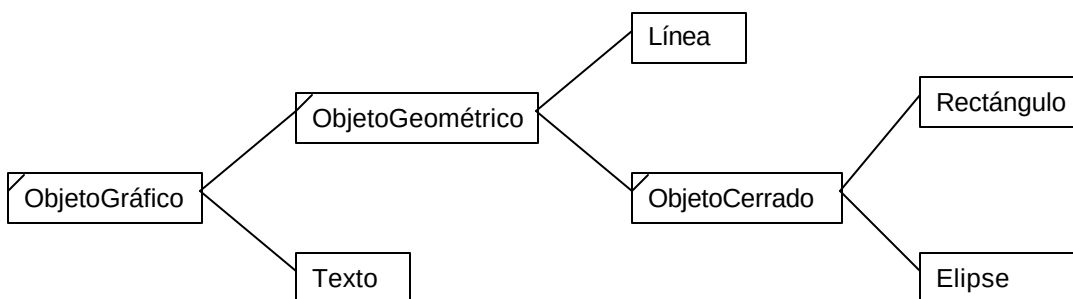
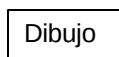
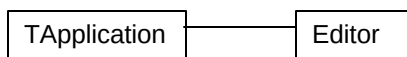
Se analiza también el esquema de colaboración entre las clases en un esfuerzo por distribuir el flujo de mensajes en forma equilibrada. Se contestan preguntas como:

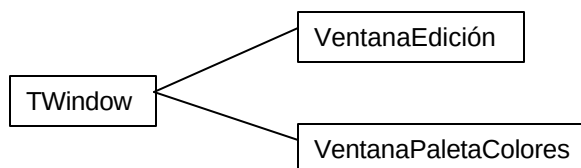
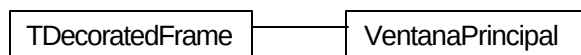
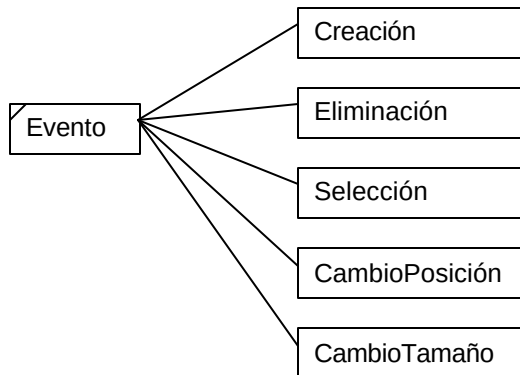
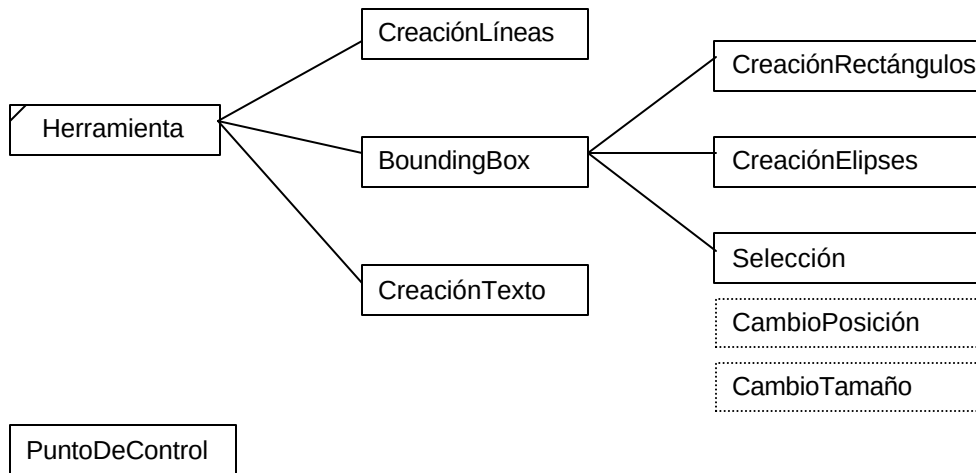
- ¿ Hay partes del sistema donde el tráfico de mensajes es particularmente intenso?
- ¿ Hay clases que colaboran con todas las demás?
- ¿ Hay clases que no colaboran con ninguna otra? (deben ser descartadas)

En las tarjetas donde se lleva la información de cada clase se agregan las superclases y subclases correspondientes.

Clase:	ObjetoCerrado	(C)
Superclases:	ObjetoGeométrico	
Subclases:	Rectángulo, Elipse	
Atributos: borde (si/no) relleno (si/no) color relleno		
Responsabilidades		
Almacenarse en disco		
Leerse de disco		

6.1 Jerarquías de herencia de la aplicación ejemplo





Con esta información es posible transformar las responsabilidades de cada clase en un conjunto de *prototipos*. Un *prototipo* es la representación formal de las entradas y salidas de un método y consiste de la información requerida para hacer uso de él: nombre del método, tipos de los parámetros que recibe, y tipo de objeto que retorna. De esta manera se tienen completamente especificadas las clases identificadas.

6.2 Prototipos de métodos de las clases de la aplicación ejemplo

0.	Editor	(concreta)
	void	InitMainWindow();
1.	Dibujo	(concreta)
	void	display(TDC& tdc);
	void	read(ifpstream&);
	void	write(ofpstream&);
	void	agregueObjeto(ObjetoGrafico*);
	void	flush();
	void	seleccione(TPoint&);
	void	seleccione(TRect&);
	void	deseleccioneTodo();
	void	seleccioneTodo();
	void	actualizarGrupo(ObjetoGrafico&);
	BOOL	dibujoVacio();
	BOOL	grupoVacio();
	BOOL	grupoContiene(const TPoint&);
	ControlPoint*	grupoPuntoDeControl(TPoint&);
	void	displayGroupBBox(TDC& , TPoint&);
	void	muevaGrupo(TPoint&);
	void	handleDoubleClick(TPoint&);
	void	actualiceFgColorGrupo(TColor&);
	void	actualiceBgColorGrupo(TColor&);
	void	actualiceBordeGrupo(BOOL);
	void	actualiceRellenoGrupo(BOOL);
2.	ObjetoGráfico	(superclase abstracta)
	void	color(TColor&);
	TColor&	color();
	void	activo(BOOL);
	BOOL	activo();
	void	boundBox(TRect&);
	TRect&	boundBox();
	void	seleccionado(BOOL);
	BOOL	seleccionado();

	void void void PuntoDeControl* void void void int	muevase(TPoint&); crearPuntosDeControl(); actualizarPuntosDeControl(); puntoDeControlEn(TPoint&); display(TDC&); showBBox(TDC& , TPoint&); showPuntosDeControl(TDC&); handleDoubleClick();	
3.	ObjetoGeométrico		(superclase abstracta)
	void BYTE void BYTE	anchoDeLinea(BYTE); anchoDeLinea(); estiloDeLinea(BYTE); estiloDeLinea();	
4.	Línea		(concreta)
	void	display(TDC&);	
5.	ObjetoCerrado		(superclase abstracta)
	void TColor& void BOOL void BOOL	colorRelleno(TColor&); colorRelleno(); borde(BOOL); borde(); relleno(BOOL); relleno();	
6.	Rectángulo		(concreta)
	void	display(TDC&);	
7.	Elipse		(concreta)
	void	display(TDC&);	
8.	Texto		(concreta)
	void int	display(TDC&); handleDoubleClick();	
9.	Herramienta		(superclase abstracta)
	void void void void void void	leftButtonDown(TPoint&); mouseMove(TPoint&); leftButtonUp(); inicieDisplay(TPoint&) = ABSTRACT; pinte(TPoint&) = ABSTRACT; termineDisplay() = ABSTRACT;	

10. herramienta CreaciónLíneas (concreta)

```
void      inicieDisplay( TPoint& );  
void      pinte( TPoint& );  
void      termineDisplay();
```

11. herramienta BoundingBox

```
void      inicieDisplay( TPoint& );  
void      pinte( TPoint& );  
void      termineDisplay();
```

12. herramienta CreaciónRectángulos (concreta)

```
void      inicieDisplay( TPoint& );  
void      pinte( TPoint& );  
void      termineDisplay();
```

13. herramienta CreaciónElipses (concreta)

```
void      inicieDisplay( TPoint& );  
void      pinte( TPoint& );  
void      termineDisplay();
```

14. herramienta de Selección (concreta)

```
void      inicieDisplay( TPoint& );  
void      pinte( TPoint& );  
void      termineDisplay();
```

15. herramienta CreaciónTextos (concreta)

```
void      inicieDisplay( TPoint& );  
void      pinte( TPoint& );  
void      termineDisplay();
```

16. herramienta CambioPosición (Selección)

```
void      inicieDisplay( TPoint& );  
void      pinte( TPoint& );  
void      termineDisplay();
```

17. herramienta CambioTamaño (Selección)

```
void      inicieDisplay( TPoint& );  
void      pinte( TPoint& );  
void      termineDisplay();
```

18. Menú (Borland)

19. Icono correspondiente a cada herramienta (Borland)

20.	Grupo de elementos seleccionados	(Borland)
21.	PuntoDeControl	(concreta)
	void muevase(TPoint&); void muevaseA(TPoint&); void resizeObjetoGráfico(TDC&, TPoint&); ObjetoGrafico* objetoGráfico(); Dirección dirección(); BOOL caeDentro(TRect&); void display(TDC&);	
22.	Evento	(superclase abstracta)
	TRect& deshacer() = ABSTRACT; TRect& rehacer() = ABSTRACT;	
23.	evento de Creación	(concreta)
	TRect& deshacer(); TRect& rehacer();	
24.	evento de Eliminación	(concreta)
	TRect& deshacer(); TRect& rehacer();	
25.	evento de Selección	(concreta)
	TRect& deshacer(); TRect& rehacer();	
26.	evento de CambioPosición	(concreta)
	TRect& deshacer(); TRect& rehacer();	
27.	evento de CambioTamaño	(concreta)
	TRect& deshacer(); TRect& rehacer();	
28.	Ventana	(Borland)
29.	Ventana principal	(Borland)
30.	Ventana de edición	(concreta)
	void Paint(TDC&, BOOL, TRect&); void cargarCursor(int);	

void	fgColor(TColor&);
TColor&	fgColor();
void	bgColor(TColor&);
TColor&	bgColor();
void	anchoLínea(BYTE);
BYTE	anchoLínea();
void	estiloLínea(BYTE);
BYTE	estiloLínea();
void	actualizarTítulo();
BOOL	borde();
BOOL	relleno();
TChooseFontDialog::TData*	getCurFont();
void	CmNuevo();
void	CmRecuperar();
void	CmAlmacenar();
void	CeAlmacenar(TCommandEnabler&);
void	CmAlmacenarComo();
void	CeAlmacenarComo(TCommandEnabler&);
void	CmBorde();
void	CeBorde(TCommandEnabler&);
void	CmRelleno();
void	CeRelleno(TCommandEnabler&);
void	CmUndo();
void	CeUndo(TCommandEnabler&);
void	CmRedo();
void	CeRedo(TCommandEnabler&);
void	CmBorrar();
void	CeBorrar(TCommandEnabler&);
void	CmCopiar();
void	CeCopiar(TCommandEnabler&);
void	CmPegar();
void	CePegar(TCommandEnabler&);
void	CmAyudaAyuda();
void	CmSelectAll();
void	CeSelectAll(TCommandEnabler&);
void	CmAcercaDeEg();
void	CmTool(WPARAM);
void	CmLine(WPARAM);
void	CmStyle(WPARAM);
void	EvLButtonDown(UINT, TPoint&);
void	EvMouseMove(UINT, TPoint&);
void	EvLButtonUp(UINT, TPoint&);
void	EvLButtonDbIClk(UINT, TPoint&);

31.	Ventana de herramientas	(concreta)
-----	-------------------------	------------

void	SetupWindow();
void	EvClose();

32.	Ventana de ancho de línea	(Borland)
-----	---------------------------	-----------

33. Ventana paleta de colores

(concreta)

```
void      Paint( TDC&, BOOL, TRect& );  
void      EvSize( UINT, TSize& );  
void      EvLButtonDown( UINT, TPoint& );  
void      EvRButtonDown( UINT, TPoint& );
```

6.3 Contratos entre las clases de la aplicación ejemplo

Una vez que se han analizado y reorganizado las jerarquías de herencia, se tiene la información necesaria para definir los contratos que soporta cada clase del sistema. Los contratos son otra herramienta de diseño, esta vez para agrupar responsabilidades funcionalmente relacionadas de una clase: permiten trabajar a un nivel de abstracción mayor, donde el sistema es menos complejo.

Un contrato define un conjunto de servicios relacionados que un objeto cliente puede solicitar a un objeto servidor. El objeto servidor debe garantizar (por medio del contrato) la prestación de esos servicios en una forma apropiada y el objeto cliente debe cumplir con el contrato solicitando solamente los servicios en él especificados. Las responsabilidades identificadas y asignadas en la fase exploratoria servirán para identificar los contratos. Una clase, entonces, puede dar soporte a uno o más contratos. Cada responsabilidad será parte a lo sumo de un contrato, existiendo responsabilidades que hacen parte del comportamiento de la clase pero que no pertenecen a ninguno de ellos. Estas últimas son referidas como responsabilidades privadas. Los criterios a seguir para la generación de contratos son los siguientes:

- i. Agrupar los servicios que son utilizados por un mismo cliente. Un contrato es un conjunto de responsabilidades cohesivas. Por lo tanto una manera de identificar un contrato soportado por una clase, está en agrupar aquellas responsabilidades que son utilizadas por un mismo cliente.
- ii. Maximizar la coherencia de las clases. Así como un contrato está compuesto por un conjunto cohesivo de responsabilidades, una clase debe soportar un conjunto cohesivo de contratos. Maximizar la cohesión de las clases tenderá a minimizar el número de contratos soportados.
- iii. Minimizar el número de contratos soportados por cada clase. Una de las claves para la prueba y mantenimiento de un sistema radica en la comprensión que se tenga del mismo. A menor cantidad de detalles a manejar, mayor comprensión.

A continuación se presenta la lista de clases y sus responsabilidades. Estas últimas han sido agrupadas en responsabilidades privadas y responsabilidades que pertenecen a un contrato. A la derecha se ha anotado la clase que hace uso del contrato.

0.	Editor	(concreta)
	void InitMainWindow();	(responsabilidad privada)

1.	Dibujo	(concreta)
	void display(TDC& tdc);	(VentanaPrincipal)
	void read(ifpstream&);	(VentanaPrincipal)

void	write(ofpstream&);	(VentanaPrincipal)
void	flush();	(VentanaPrincipal)
void	handleDoubleClick(TPoint&);	(VentanaPrincipal)
void	agregueObjeto(ObjetoGrafico*);	(VentanaPrincipal)
void	seleccione(TPoint&);	(VentanaPrincipal)
void	seleccione(TRect&);	(VentanaPrincipal)
void	seleccioneTodo();	(VentanaPrincipal)
void	deseleccioneTodo();	(VentanaPrincipal)

Contrato 1: manipular el dibujo

void	displayGroupBBox(TDC& , TPoint&);	(VentanaPrincipal)
void	muevaGrupo(TPoint&);	(VentanaPrincipal)
void	actualiceFgColorGrupo(TColor&);	(VentanaPrincipal)
void	actualiceBgColorGrupo(TColor&);	(VentanaPrincipal)
void	actualiceBordeGrupo(BOOL);	(VentanaPrincipal)
void	actualiceRellenoGrupo(BOOL);	(VentanaPrincipal)

Contrato 2: manipular el grupo de elementos seleccionados

BOOL	dibujoVacio();	(VentanaPrincipal)
BOOL	grupoVacio();	(VentanaPrincipal)
BOOL	grupoContiene(const TPoint&);	(VentanaPrincipal)
ControlPoint*	grupoPuntoDeControl(TPoint&);	(VentanaPrincipal)

Contrato 3: retornar información sobre el estado del dibujo y del grupo

void	actualizarGrupo(ObjetoGrafico&);	(responsabilidad privada)
------	------------------------------------	---------------------------

2. ObjetoGráfico (superclase abstracta)

void	color(TColor&);	(Dibujo)
void	activo(BOOL);	(Dibujo)
void	seleccionado(BOOL);	(Dibujo)
void	muevase(TPoint&);	(Dibujo)
int	handleDoubleClick();	(Dibujo)
void	boundBox(TRect&);	(PuntoDeControl)

Contrato 4: cambiar atributos

BOOL	seleccionado();	(Dibujo)
BOOL	activo();	(Dibujo)
PuntoDeControl*	puntoDeControlEn(TPoint&);	(Dibujo)

Contrato 5: dar información sobre atributos

void	display(TDC&);	(Dibujo)
void	showPuntosDeControl(TDC&);	(PuntoDeControl)
void	showBBox(TDC& , TPoint&);	(HerramientaSelección)

Contrato 6: mostrar atributos

TColor&	color();	(responsabilidad privada)
TRect&	boundBox();	(responsabilidad privada)
void	crearPuntosDeControl();	(responsabilidad privada)
void	actualizarPuntosDeControl();	(responsabilidad privada)

3. ObjetoGeométrico (superclase abstracta)

void	anchoDeLinea(BYTE);	(Dibujo)
void	estiloDeLinea(BYTE);	(Dibujo)
BYTE	anchoDeLinea();	(responsabilidad privada)
BYTE	estiloDeLinea();	(responsabilidad privada)

4. Línea (concreta)

void	display(TDC&);	(Dibujo)
------	------------------	----------

5. ObjetoCerrado (superclase abstracta)

void	colorRelleno(TColor&);	(Dibujo)
void	borde(BOOL);	(Dibujo)
void	relleno(BOOL);	(Dibujo)
TColor&	colorRelleno();	(responsabilidad privada)
BOOL	borde();	(responsabilidad privada)
BOOL	relleno();	(responsabilidad privada)

6. Rectángulo (concreta)

void	display(TDC&);	(Dibujo)
------	------------------	----------

7. Elipse (concreta)

void	display(TDC&);	(Dibujo)
------	------------------	----------

8. Texto (concreta)

void	display(TDC&);	(Dibujo)
int	handleDoubleClick();	(Dibujo)

9. Herramienta (superclase abstracta)

void	leftButtonDown(TPoint&);	(VentanaPrincipal)
void	mouseMove(TPoint&);	(VentanaPrincipal)
void	leftButtonUp();	(VentanaPrincipal)

Contrato 7: responder a eventos de mouse

void	inicieDisplay(TPoint&) = ABSTRACT;	(responsabilidad privada)
void	pinte(TPoint&) = ABSTRACT;	(responsabilidad privada)
void	termineDisplay() = ABSTRACT;	(responsabilidad privada)

10. herramienta CreaciónLíneas (concreta)

void	inicieDisplay(TPoint&);	(responsabilidad privada)
void	pinte(TPoint&);	(responsabilidad privada)
void	termineDisplay();	(responsabilidad privada)

11. herramienta BoundingBox

void	inicieDisplay(TPoint&);	(responsabilidad privada)
void	pinte(TPoint&);	(responsabilidad privada)
void	termineDisplay();	(responsabilidad privada)

12. herramienta CreaciónRectángulos (concreta)

void	inicieDisplay(TPoint&);	(responsabilidad privada)
void	pinte(TPoint&);	(responsabilidad privada)
void	termineDisplay();	(responsabilidad privada)

13. herramienta CreaciónElipses (concreta)

void	inicieDisplay(TPoint&);	(responsabilidad privada)
void	pinte(TPoint&);	(responsabilidad privada)
void	termineDisplay();	(responsabilidad privada)

14. herramienta de Selección (concreta)

void	inicieDisplay(TPoint&);	(responsabilidad privada)
void	pinte(TPoint&);	(responsabilidad privada)
void	termineDisplay();	(responsabilidad privada)

15. herramienta CreaciónTextos (concreta)

void	inicieDisplay(TPoint&);	(responsabilidad privada)
void	pinte(TPoint&);	(responsabilidad privada)
void	termineDisplay();	(responsabilidad privada)

16. herramienta CambioPosición (Selección)

void	inicieDisplay(TPoint&);	(responsabilidad privada)
void	pinte(TPoint&);	(responsabilidad privada)
void	termineDisplay();	(responsabilidad privada)

17. herramienta CambioTamaño (Selección)

	void void void	inicieDisplay(TPoint&); pinte(TPoint&); termineDisplay();	(responsabilidad privada) (responsabilidad privada) (responsabilidad privada)
18.	Menú		(Borland)
19.	Icono correspondiente a cada herramienta		(Borland)
20.	Grupo de elementos seleccionados		(Borland)
21.	PuntoDeControl		(concreta)
	void void	muevase(TPoint&); muevaseA(TPoint&);	(ObjetoGráfico) (ObjetoGráfico)
Contrato 8: cambiar atributos			
	BOOL ObjetoGrafico*	caeDentro(TRect&); objetoGráfico();	(Dibujo) (HerramientaSelección)
Contrato 9: retornar información sobre atributos			
	void	resizeObjetoGráfico(TDC&, TPoint&);	(HerramientaSelección)
Contrato 10: cambiar atributos del elemento que lo contiene			
	void	display(TDC&);	(Dibujo)
Contrato 11: mostrar atributos			
	Dirección	dirección();	(sobra)
22.	Evento		(superclase abstracta)
	TRect& TRect&	deshacer() = ABSTRACT; rehacer() = ABSTRACT;	(VentanaPrincipal) (VentanaPrincipal)
Contrato 12: cambiar atributos del dibujo			
23.	evento de Creación		(concreta)
	TRect& TRect&	deshacer(); rehacer();	(VentanaPrincipal) (VentanaPrincipal)
24.	evento de Eliminación		(concreta)

	TRect& TRect&	deshacer(); rehacer();	(VentanaPrincipal) (VentanaPrincipal)
25.	evento de Selección		(concreta)
	TRect& TRect&	deshacer(); rehacer();	(VentanaPrincipal) (VentanaPrincipal)
26.	evento de CambioPosición		(concreta)
	TRect& TRect&	deshacer(); rehacer();	(VentanaPrincipal) (VentanaPrincipal)
27.	evento de CambioTamaño		(concreta)
	TRect& TRect&	deshacer(); rehacer();	(VentanaPrincipal) (VentanaPrincipal)
28.	Ventana		(Borland)
29.	Ventana principal		(Borland)
30.	Ventana de edición		(concreta)
	void	Paint(TDC&, BOOL, TRect&);	(Windows)
	void	CmNuevo();	(usuario)
	void	CmRecuperar();	(usuario)
	void	CmAlmacenar();	(usuario)
	void	CeAlmacenar(TCommandEnabler&);	(usuario)
	void	CmAlmacenarComo();	(usuario)
	void	CeAlmacenarComo(TCommandEnabler&);	(usuario)
	void	CmBorde();	(usuario)
	void	CeBorde(TCommandEnabler&);	(usuario)
	void	CmRelleno();	(usuario)
	void	CeRelleno(TCommandEnabler&);	(usuario)
	void	CmUndo();	(usuario)
	void	CeUndo(TCommandEnabler&);	(usuario)
	void	CmRedo();	(usuario)
	void	CeRedo(TCommandEnabler&);	(usuario)
	void	CmBorrar();	(usuario)
	void	CeBorrar(TCommandEnabler&);	(usuario)
	void	CmCopiar();	(usuario)
	void	CeCopiar(TCommandEnabler&);	(usuario)
	void	CmPegar();	(usuario)
	void	CePegar(TCommandEnabler&);	(usuario)
	void	CmAyudaAyuda();	(usuario)
	void	CmSelectAll();	(usuario)
	void	CeSelectAll(TCommandEnabler&);	(usuario)
	void	CmAcercaDeEg();	(usuario)
	void	EvLButtonDown(UINT, TPoint&);	(usuario)

void	EvMouseMove(UINT, TPoint&);	(usuario)
void	EvLButtonUp(UINT, TPoint&);	(usuario)
void	EvLButtonDbIClk(UINT, TPoint&);	(usuario)

Contrato 13: procesar eventos de usuario (a través de Windows)

void	fgColor(TColor&);	(VentanaPaletaColores)
void	bgColor(TColor&);	(VentanaPaletaColores)
void	CmTool(WPARAM);	(VentanaHerramientas)
void	CmLine(WPARAM);	(VentanaLíneas)
void	CmStyle(WPARAM);	(VentanaLíneas)

Contrato 14: cambiar atributos

TColor&	fgColor();	(Herramienta)
TColor&	bgColor();	(Herramienta)
BYTE	anchoLínea();	(Herramienta)
BYTE	estiloLínea();	(Herramienta)
BOOL	borde();	(Herramienta)
BOOL	relleno();	(Herramienta)
TChooseFontDialog::TData*	getCurFont();	(Herramienta)

Contrato 15: retornar información sobre atributos

void	actualizarTítulo();	(responsabilidad privada)
void	cargarCursor(int);	(responsabilidad privada)
void	anchoLínea(BYTE);	(sobra)
void	estiloLínea(BYTE);	(sobra)

31. Ventana de herramientas (concreta)

void	SetupWindow();	(responsabilidad privada)
void	EvClose();	(responsabilidad privada)

32. Ventana de ancho de línea (Borland)

33. Ventana paleta de colores (concreta)

void	Paint(TDC&, BOOL, TRect&);	(Windows)
void	EvSize(UINT, TSize&);	(responsabilidad privada)
void	EvLButtonDown(UINT, TPoint&);	(responsabilidad privada)
void	EvRButtonDown(UINT, TPoint&);	(responsabilidad privada)

7. Subsistemas

Los pasos anteriores estaban encaminados a producir un diseño inicial. Se identificaron las clases del sistema, sus responsabilidades y las colaboraciones que garantizarán los servicios requeridos. Se analizaron las relaciones de herencia entre las clases y se identificaron los contratos soportados. En esta última fase se refina el diseño obtenido simplificando los patrones de comunicación entre las clases. Se dividen las responsabilidades en grupos de clases llamados *subsistemas* y se mejora la forma en que estos encapsulan la funcionalidad global. Luego se diseñan interfaces con estos subsistemas de tal forma que simplifiquen el flujo de información y de control.

Para simplificar el diseño inicial se cuenta con dos herramientas adicionales:

grafos de colaboraciones, y
tarjetas para el registro de subsistemas

En pasos anteriores se destacó la importancia de colocar el sistema en diferentes escenarios ('walk-throughs'): simular un conjunto de entradas plausibles, analizar el flujo de información y de control ante este conjunto de entradas y validar las respuestas con los requerimientos establecidos. Si bien cada uno de estos escenarios nos descubre un camino por donde fluye la información, no existe un escenario único que nos muestre todos los caminos posibles. Sin embargo, para analizar en forma exhaustiva las colaboraciones existentes entre las clases, se necesita la descripción de *todos* los caminos posibles. Un grafo de colaboraciones es la herramienta que nos permite describir y analizar todos estos caminos.

Los grafos de colaboraciones representan las clases, los contratos y las colaboraciones existentes. Pueden ser utilizados para identificar áreas de complejidad innecesaria, duplicación, o lugares donde el encapsulamiento es violado. A partir de un grafo de colaboraciones completo para la aplicación se pueden identificar subsistemas, simplificar los patrones de comunicación y en consecuencia lograr un diseño más claro y comprensible.

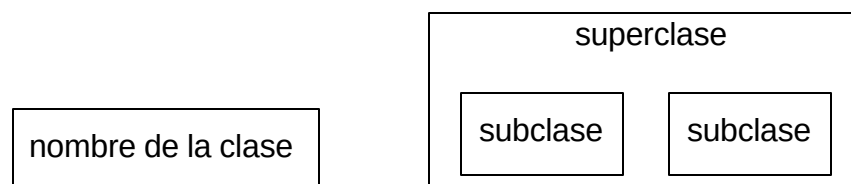


Fig. 1. Representación de clases en grafos de colaboraciones.

La figura 6 muestra la representación de una clase en un grafo de colaboraciones y también se puede ver la relación superclase-subclase. Se debe tener en cuenta que una subclase soporta todos los contratos definidos por sus superclases. Como vimos anteriormente, las responsabilidades de una clase están agrupadas en contratos en los cuales la clase actúa como servidor. Estos contratos se representan con un semicírculo numerado:

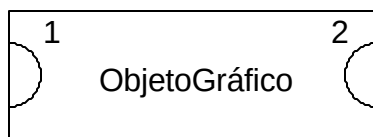


Fig. 2. Representación de contratos.

Las colaboraciones entre las clases se muestran mediante una flecha que va de la clase cliente al contrato soportado por la clase servidora.

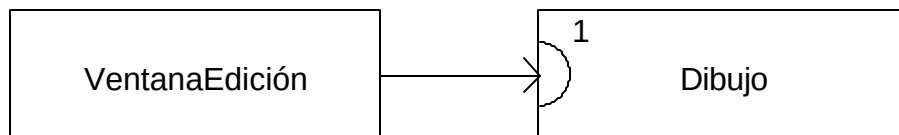


Fig. 3. Representación de colaboraciones.

Los subsistemas están compuestos por clases (o grupos de clases y otros subsistemas) que colaboran entre sí para dar soporte a un grupo de contratos. Desde afuera, un subsistema es visto como un conjunto de clases que colaboran para proveer cierta funcionalidad claramente delimitada del sistema. Por dentro, los subsistemas revelan una estructura compleja. Realmente no existe una diferencia conceptual entre las responsabilidades de una clase, de un subsistema e inclusive de una aplicación completa: es simplemente una cuestión de escala.

Los subsistemas, al igual que las clases, soportan contratos. Para determinar los contratos soportados por un subsistema, se buscan las clases que hacen parte de él y que prestan servicios a clases clientes por fuera del subsistema. Una clase hace parte de un subsistema sólo si existe con el único propósito de cumplir con los objetivos del subsistema.

Una vez identificados, los subsistemas se registran en tarjetas similares a las utilizadas para las clases (fig. 4). Se registra el nombre dado al subsistema, al respaldo su propósito general, y se registran los contratos soportados. Dado que un subsistema es una entidad conceptual (no existe en tiempo de ejecución) no puede dar soporte directo a los contratos: a la derecha de cada contrato se anota entonces la clase que efectivamente le da soporte, esto es, la clase en la cual el subsistema delega esa responsabilidad. En la figura 4 también se muestra la representación utilizada para los subsistemas en los grafos de colaboraciones.

Subsistema: nombre	
Contrato	Delegado en

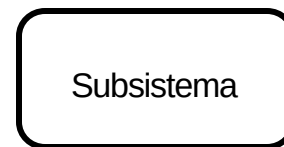


Fig. 4. Subsistemas.

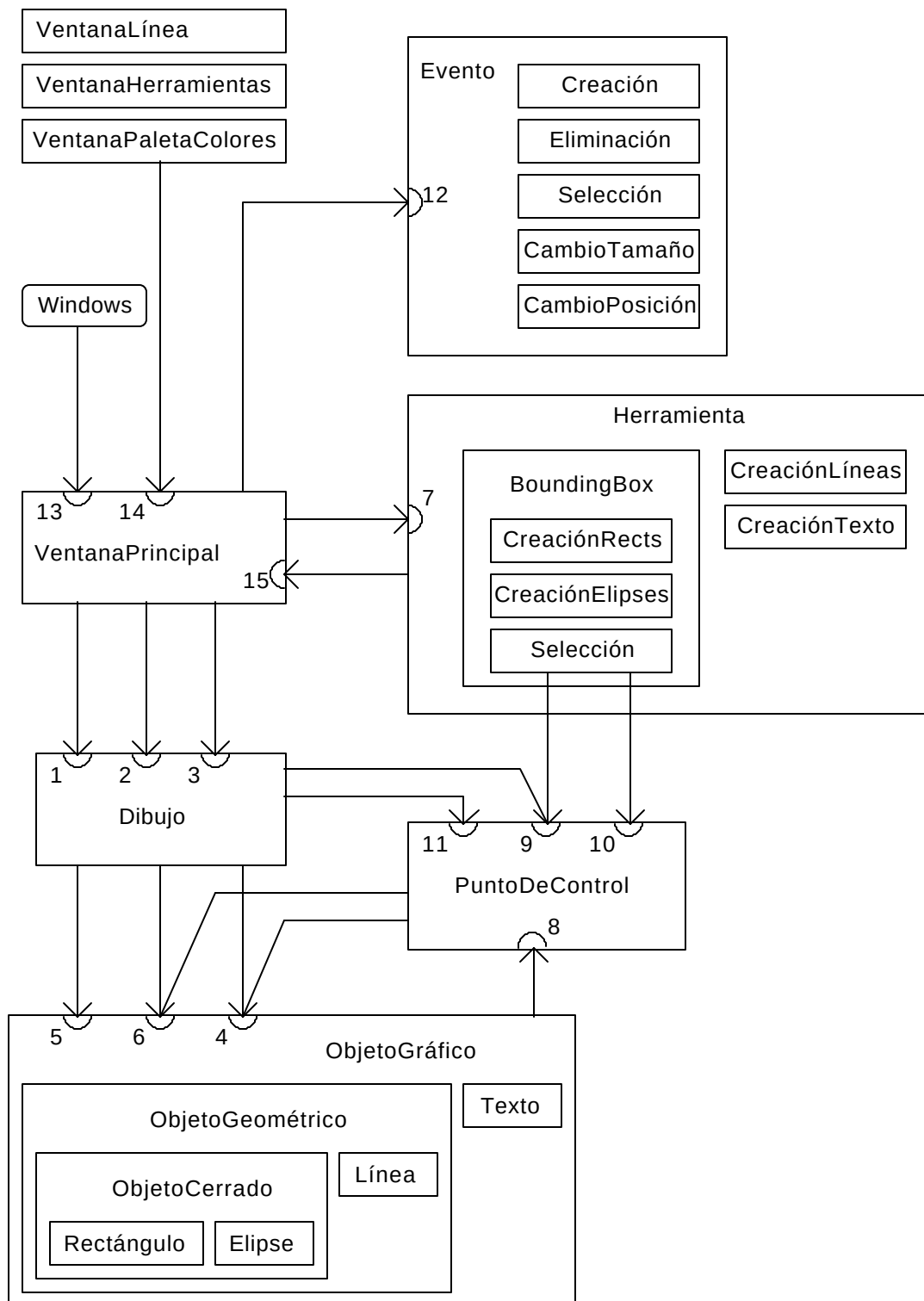
A partir de lo expuesto, los autores proponen los siguientes pasos:

- ### Construir un grafo de colaboraciones completo para la aplicación,
- ### Identificar y registrar los subsistemas existentes,
- ### Simplificar las colaboraciones entre subsistemas y dentro de los subsistemas (entre clases).

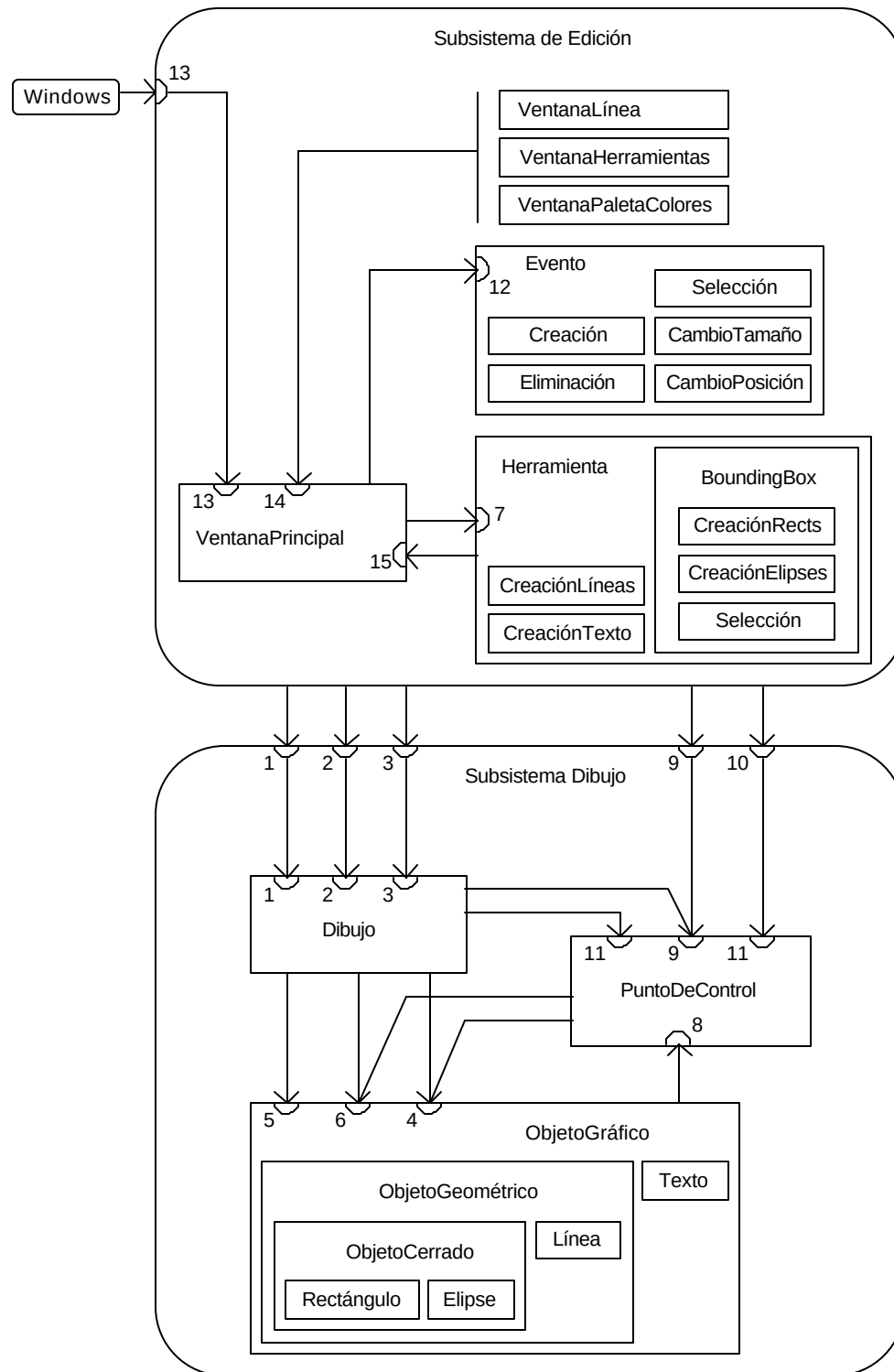
Para esto se debe:

- ### Minimizar el número de colaboraciones en el que cada clase participa,
- ### Minimizar el número de clases (y subsistemas) en el que cada subsistema delega sus responsabilidades,
- ### Minimizar el número de contratos soportado por cada clase o subsistema.

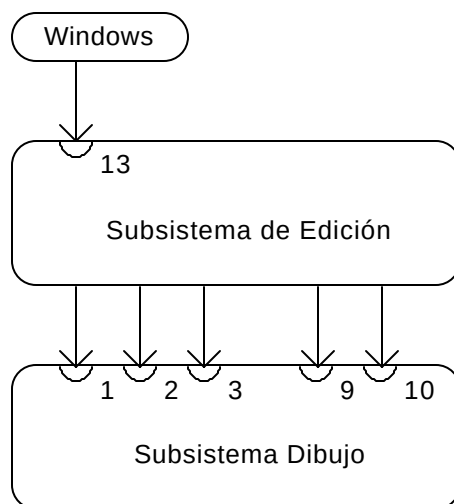
7.1 Diagrama de colaboraciones de la aplicación ejemplo



7.2 Subsistemas de la aplicación ejemplo



Subsistemas: diagrama de bloques



Subsistema de Edición	
Contrato	Delegado en
13. Procesar eventos de usuario (Windows)	VentanaPrincipal

Subsistema Dibujo	
Contrato	Delegado en
1. Manipular el dibujo	Dibujo
2. Manipular el grupo de elementos seleccionados	Dibujo
3. Retornar información sobre el estado del dibujo y del grupo	Dibujo
4. Retornar información sobre puntos de control	PuntoDeControl
5. Cambiar atributos de un elemento	PuntoDeControl

8. Protocolos

El último paso consiste en utilizar la información generada por los procesos anteriores y producir la especificación de diseño final. Para esto se proponen las siguientes actividades:

1. Definir los protocolos de cada clase.

El protocolo de una clase está compuesto por los prototipos de los métodos de la clase.

2. Escribir la especificación final de diseño para cada clase.

Esta debe incluir:

- ### Nombre y tipo (abstracta o concreta)
- ### Lista de sus superclases y subclases inmediatas
- ### Referencias a su ubicación en las jerarquías de clases
- ### Referencias a su ubicación en los grafos de colaboración
- ### Descripción de su propósito y utilización
- ### Lista de cada uno de los contratos donde actúa como clase servidora, incluir contratos heredados
- ### Para cada contrato, la lista de responsabilidades que lo soportan. Debajo de cada responsabilidad escribir las rúbricas de los métodos que la implantan.
- ### Lista de las responsabilidades privadas
- ### Consideraciones de implantación: algoritmos, restricciones, condiciones de error, excepciones, etc.

3. Escribir la especificación final de diseño para cada subsistema.

Esta debe incluir:

- ### Nombre
- ### Lista de las clases y subsistemas incluidos
- ### Referencias a su ubicación en los grafos de colaboración
- ### Descripción de su propósito
- ### Lista de cada uno de los contratos donde actúa como servidor
- ### Para cada contrato, identificar la clase (o subsistema) en el cual se delega la responsabilidad.

4. Escribir la especificación de diseño para cada contrato.

Incluye el nombre y número del contrato, servidor(es), clientes y una descripción del mismo.

9. Consideraciones de implantación

La primera consideración (siempre que se pueda elegir) se refiere al lenguaje de programación a utilizar para la implantación del diseño realizado. Si bien existen muchos lenguajes disponibles, es claro que después de haber llevado a cabo una etapa de análisis y una etapa de diseño orientadas por objetos, lo más apropiado es utilizar un lenguaje orientado por objetos. Dentro de los lenguajes orientados por objetos hay dos enfoques fundamentales: los lenguajes puros y los lenguajes híbridos.

En un lenguaje puro todo es un objeto: Smalltalk y Eiffel son ejemplos de éstos. En un lenguaje híbrido se pueden definir y utilizar objetos, pero también se pueden utilizar tipos intrínsecos (char, int, float, ...) que no son objetos. C++, Objective-C, Common LISP y Pascal OO, son ejemplos de estos lenguajes. El utilizar un lenguaje puro permite una implantación más limpia del diseño, pero en general implica un programa ejecutable mucho más grande. Un lenguaje híbrido tiene la desventaja de que los tipos intrínsecos que provee no son extendibles, ya que no son clases. Por lo general, sin embargo, estos lenguajes generan programas más eficientes con respecto a velocidad de ejecución, manejo de memoria y tamaño del ejecutable.

Ya sea que se utilice un lenguaje puro o híbrido, éste debe proveer por lo menos las siguientes características:

- clases como objetos
- herencia simple y múltiple
- polimorfismo (encadenamiento tardío)
- verificación de tipos en tiempo de compilación
- información de tipos en tiempo de ejecución
- manejo automático de memoria
- manejo de excepciones

Adicionalmente un buen lenguaje de programación debe proveer:

- compilación incremental
- un entorno de programación integrado (editor, compilador, encadenador, depurador, browser de clases, etc.)
- una buena biblioteca de clases
- un Application Framework consistente

9.1 Algunas consideraciones sobre la implantación de clases abstractas

En secciones anteriores se mencionó que el propósito de las clases abstractas está en encapsular el comportamiento común a un grupo de clases. Ahora se hace una diferenciación entre los métodos que una clase abstracta puede declarar.

Existen *métodos básicos* que proveen un comportamiento que es útil a todas las clases descendientes. Un ejemplo se encuentra en la clase ObjetoGráfico que provee un método que retorna el bounding box del objeto. Este método es heredado por todas las clases concretas descendientes de ObjetoGráfico sin necesidad de ser redefinido.

Están los *métodos abstractos* que especifican un comportamiento que todas las clases descendientes deben tener. Estos métodos son declarados en la clase abstracta, pero no definidos: deben ser definidos en todas las clases concretas descendientes. Un ejemplo de esto también se encuentra en la clase ObjetoGráfico que declara un método *display()*. Este método es declarado en ObjetoGráfico como un método abstracto (el compilador provee el mecanismo para esto) y de esta manera se obliga a todas las clases concretas descendientes a definirlo.

Y también están los *métodos prototipos* donde se especifica un algoritmo paso a paso pero sin definirlo completamente. Un ejemplo de esto está en la clase abstracta Herramienta que provee el comportamiento prototipo para procesar los eventos de mouse LButtonDown, MouseMove y LButtonUp. Cuando sucede un evento LButtonDown la clase Herramienta almacena las coordenadas del punto donde ocurrió el evento. En seguida llama a un método *inicieDisplay()* encargado justamente de comenzar a pintar. Como esto depende de la herramienta concreta en cuestión, este método es declarado abstracto en Herramienta para que obligatoriamente sea definido en las clases descendientes. Lo mismo sucede en los métodos de Herramienta que procesan los eventos MouseMove y LButtonUp, donde se registra la posición del mouse y luego se llaman los métodos abstractos *pinte()* y *termineDisplay()*.

9.2 Pruebas del diseño

Dado que se han seguido los principios de ocultamiento de información y encapsulamiento, resulta fácil probar cada parte del sistema en forma independiente. Si se ha encapsulado el comportamiento del sistema en clases cuyas interfaces se han controlado estrictamente, entonces se puede estar seguro de cada parte (clase o subsistema) no afecta al resto sino en formas conocidas y por lo tanto probables. Cuando se tiene una clase probada, se puede agregar al sistema sin temor de que aparezcan fallas en otras partes aparentemente sin relación. De la misma forma, cuando una clase o subsistema necesita mantenimiento, se pueden hacer los cambios necesarios con la seguridad de que éstos no tendrán efectos laterales en el resto del sistema.

Con respecto a las jerarquías de clases se recomienda comenzar a probar las superclases. De esta manera cuando se va a probar una subclase ya se ha probado el comportamiento heredado. Para probar una subclase se debe primero validar que soporte apropiadamente los contratos definidos por sus superclases, y luego los nuevos contratos por ella definidos.

9.3 Midiendo la calidad del diseño

Es muy difícil evaluar la calidad de un diseño, máxime cuando no se tiene un punto de comparación. Sin embargo puede ayudar el responder las siguientes preguntas:

¿Cuántas clases tiene el diseño final?

Un sistema que tiene muchas clases implica que las clases que lo componen encapsulan relativamente poca inteligencia y que la arquitectura del mismo es compleja. Un número pequeño de clases más inteligentes generalmente implica un diseño más sencillo. Sin embargo, cuanto más inteligencia encapsula una clase, más difícil resulta su reutilización.

¿Cuántos subsistemas tiene el diseño final?

En general, a mayor cantidad de subsistemas encapsulando la funcionalidad del sistema global, mejor.

¿Cuántos contratos hay por clase?

El número promedio de contratos por clase es otro indicador de la complejidad del diseño producido. Por lo general, a mayor cantidad de contratos por clase, más difícil de mantener la aplicación.

¿Cuántas clases abstractas tiene?

Un número grande de clases abstractas, que es equivalente a jerarquías de herencia de muchos niveles, es índice de que tanto se ha refinado el diseño. Jerarquías de herencia de muchos niveles significan más código reutilizable.

9.4 Final

Por último se debe tener en cuenta que la implantación de un programa conlleva ciertas circunstancias contradictorias. Por un lado se debe cumplir un plazo estipulado. Por otro se deben asimilar los cambios en la especificación de requerimientos, que nunca faltan. Y por otro, se debe preservar la esencia del diseño. Frente a esta situación hay que recordar que no existe un diseño perfecto y que en tiempo de implantación siempre van a aparecer ciertos detalles que no fueron considerados en el diseño. La experiencia, sin duda, juega un papel fundamental.

Referencias

- [Coad] Coad, P., Yourdon, E., *Análise Baseada em Objetos*. Editora Campus, Rio de Janeiro, 1992.
- [Jackson] Jackson, M. A., *Principles of Program Design*. Academic Press, London, 1975.
- [Jackson] Jackson, M. A., *System Development*. Prentice-Hall, London, 1983.
- [Meyer] Meyer, B., *Object-Oriented Software Construction*. Prentice-Hall, Hemel Hempstead, 1988.
- [Sommerville] Sommerville, I., *Software Engineering*. Addison-Wesley, Wokingham, 1989.
- [Wirfs-Brock] Wirfs-Brock, R., Wilkerson, B. y Wiener, L., *Designing Object-Oriented Software*, Prentice Hall, Englewood Cliffs, 1990.